题型

选择题：30` 单选

综合题：40` 关系代数、sql、画er图

简答题：30` nosql和sql最大的差异

第一章 绪论

01 管理数据的方式

1. 数据管理有几种方式？特点是什么？

- 人工管理
- 文件系统
- 数据库系统

	人工管理	**文件系统**	**数据库系统**
数据的管理者	用户（程序员），数据不保存	文件系统，数据可长期**保存**	数据库管理系统，数据可长期保存
数据面向的对象	某一应用程序	某一应用	某一应用
数据的共享程度	无共享、冗余度极大	共享性差、冗余度大	可为各种用户共享，冗余度较小
数据的独立性	不独立，完全依赖于程序	独立性差	独立性较高
数据的结构化	**无结构**	记录内有结构，整体无结构	数据结构化（**整体结构化**，数据用数据模型描述）
数据控制能力	应用程序自己控制	应用程序自己控制	数据库管理系统控制

2. 为什么要使用数据库？

- 可以负责**定义和管理**这些数据
    - 确保**数据结构和存储机制**对于所有应用来说都是可以接受的
    - 确保数据安全性，完整性
    - 解决数据的**并发**
    - 在动态条件下解决上述问题

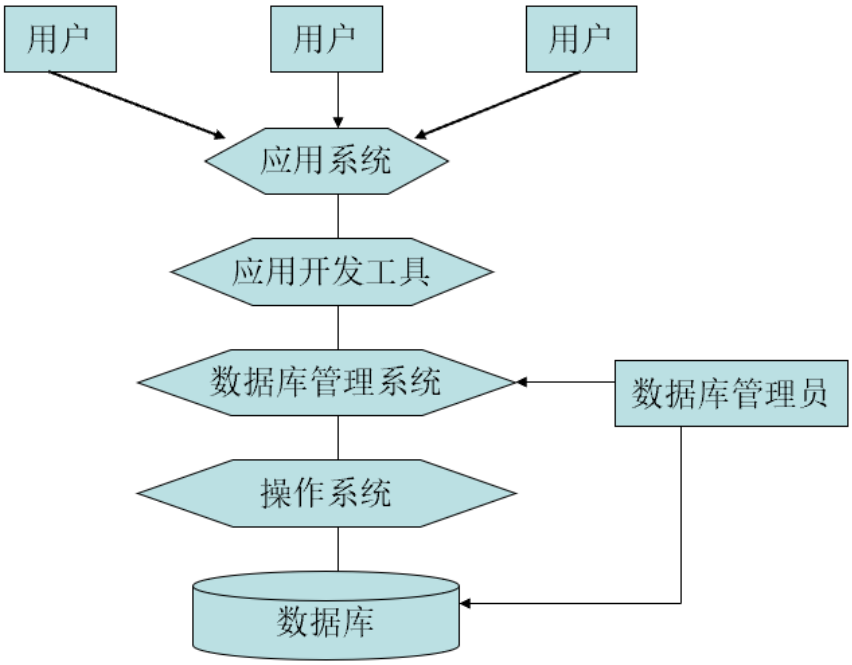
02 数据库相关概念

1. 基础概念：

- 数据：描述事物的符号记录。数据的含义称为数据的语义，数据与其**语义**是不可分的。
- 数据库：数据库（Database）是**长期**储存在计算机内、有**组织**的、可**共享**的**大量数据**的集合。
    - 特点（5）：

        - 数据按一定的数据模型**组织、描述和储存**
        - 可为各种用户**共享**
        - 冗余度较小
        - 数据独立性较高
        - 易扩展
- 数据库管理系统：位于**用户与操作系统之间**的一层**数据管理软件**（基础软件）
    - 功能（6）：选择题选
        - 数据定义功能（DDL）
        - 数据组织、存储和管理
        - 数据操纵功能（DML）
        - 数据库的事务管理和运行管理
        - 数据库的建立和维护功能
        - 其它功能，如不同数据库之间的互访和互操作功能。
- 数据库系统（见07）：由
    - 数据库（数据结构、数据项，值和形）
    - 数据库管理系统（及其应用开发工具，如DMLDDL编译器）
    - 应用程序
    - 数据库管理员（DBA） 四方面构成。



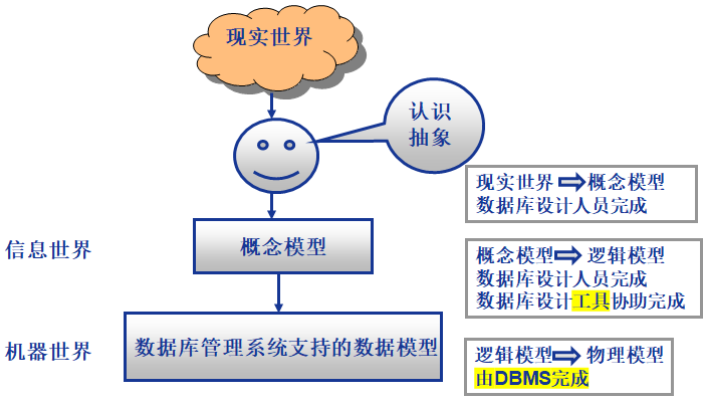


2. 数据库的主要功能？数据库用来做什么？数据库有什么特点？（4）
    1. 数据结构化
        - **整体结构化**（与文件系统的本质区别）
        - 数据用数据模型描述
    2. 数据的共享性高，冗余度低且易扩充
    3. 数据独立性高：物理存储+逻辑结构
    4. 数据由数据管理系统统一**管理和控制**
        - 安全性保护
        - 完整性检查

- 并发控制
- 数据库恢复

03 模型

1. 数据模型的定义是什么？有什么组成要素？
    1. 数据模型是对现实世界数据特征的抽象，用以**抽象、表示和处理**现实世界中的数据和信息
    2. 有3个组成要素：
        - 数据结构（刻画模型性质的最基本方面）
        - 数据操作（查询+更新）
        - 数据的完整性约束条件
2. 概念/逻辑/物理模型分别完成的任务？功能？

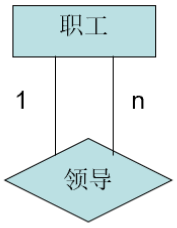


- 概念模型：
    - 按**用户**的观点来对数据和信息建模
        - 把现实世界中的客观对象变成**E-R模型**
- 逻辑模型：
    - 按**计算机系统**的观点对数据建模，用于DBMS实现
        - 决定如何抽象数据结构和组织存放数据（比如用NoSQL，表）
- 物理模型：
    - 对数据最底层的抽象，描述数据在**系统内部**的表示方式和存取方法。
        - 和环境、部署、数据库系统挂钩（如MySQL）

04 概念模型

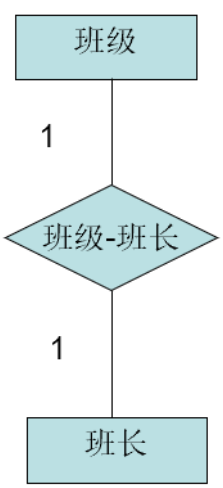
1. 概念模型的定义和组成？
    - 按**用户的观点**来对数据和信息建模，用于数据库设计（E-R图表示）
    - 实体
    - 属性：注意**实体和联系**都可以有属性！
    - 码：唯一标识实体
    - 实体型：**实体名+属性名**，如学生（学号，姓名，性别）是一个实体型
    - 实体集：同一类型实体的集合
    - 联系

- 组成实体的各属性之间的联系（内部）

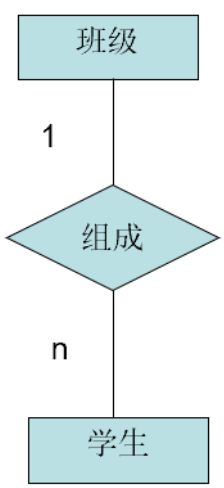


同一实体型内部的
1:n联系

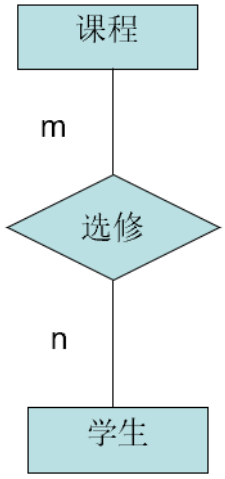
- 不同实体集之间的联系（之间）
  - 一对一
  - 一对多
  - 多对多



1:1联系



1:n联系



m:n联系

2. 如何画E-R模型?

- 物资管理涉及以下几个实体。
  - 仓库：属性有仓库号、面积、电话号码；
  - 零件：属性有零件号、名称、规格、单价、描述；
  - 供应商：属性有供应商号、姓名、地址、电话号码、账号；
  - 项目：属性有项目号、预算、开工日期；
  - 职工：属性有职工号、姓名、年龄、职称。

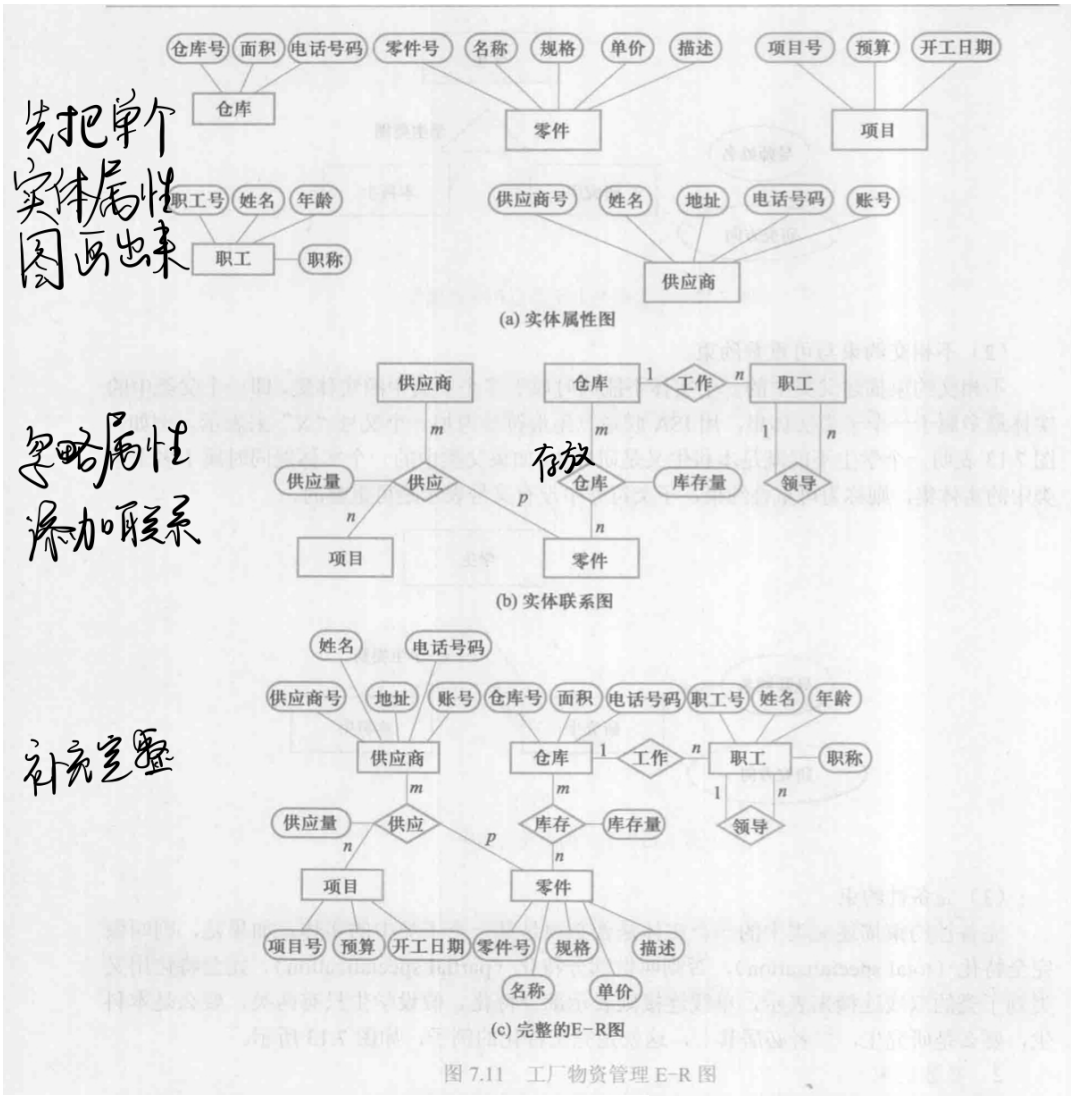
  这些实体之间的联系如下：

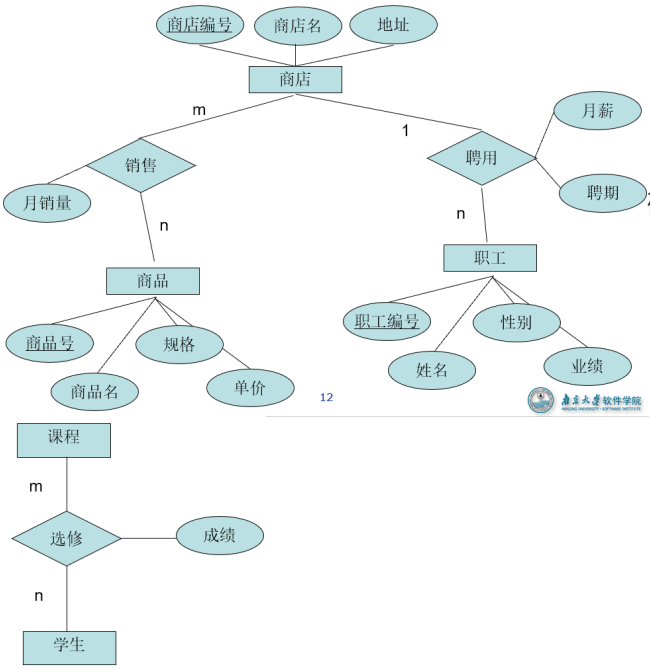
  （1）一个仓库可以存放多种零件，一种零件可以存放在多个仓库中，因此仓库和零件具有多对多的联系。用库存量来表示某种零件在某个仓库中的数量。

  （2）一个仓库有多个职工当仓库保管员，一个职工只能在一个仓库工作，因此仓库和职工之间是一对多的联系。

  （3）职工之间具有领导与被领导关系，即仓库主任领导若干保管员，因此职工实体型中具有一对多的联系。

  （4）供应商、项目和零件三者之间具有多对多的联系，即一个供应商可以供给若干项目多种零件，每个项目可以使用不同供应商供应的零件，每种零件可由不同供应商供给。下面给出此工厂的物资管理 E-R 图，如图 7.11 所示。其中，图 7.11(a)为实体属性图，图 7.11(b)为实体联系图，图 7.11(c)为完整的 E-R 图。这里把实体的属性单独画出仅仅是为了更清晰地表示实体及实体之间的联系。

- 

- 

码下方有下划线

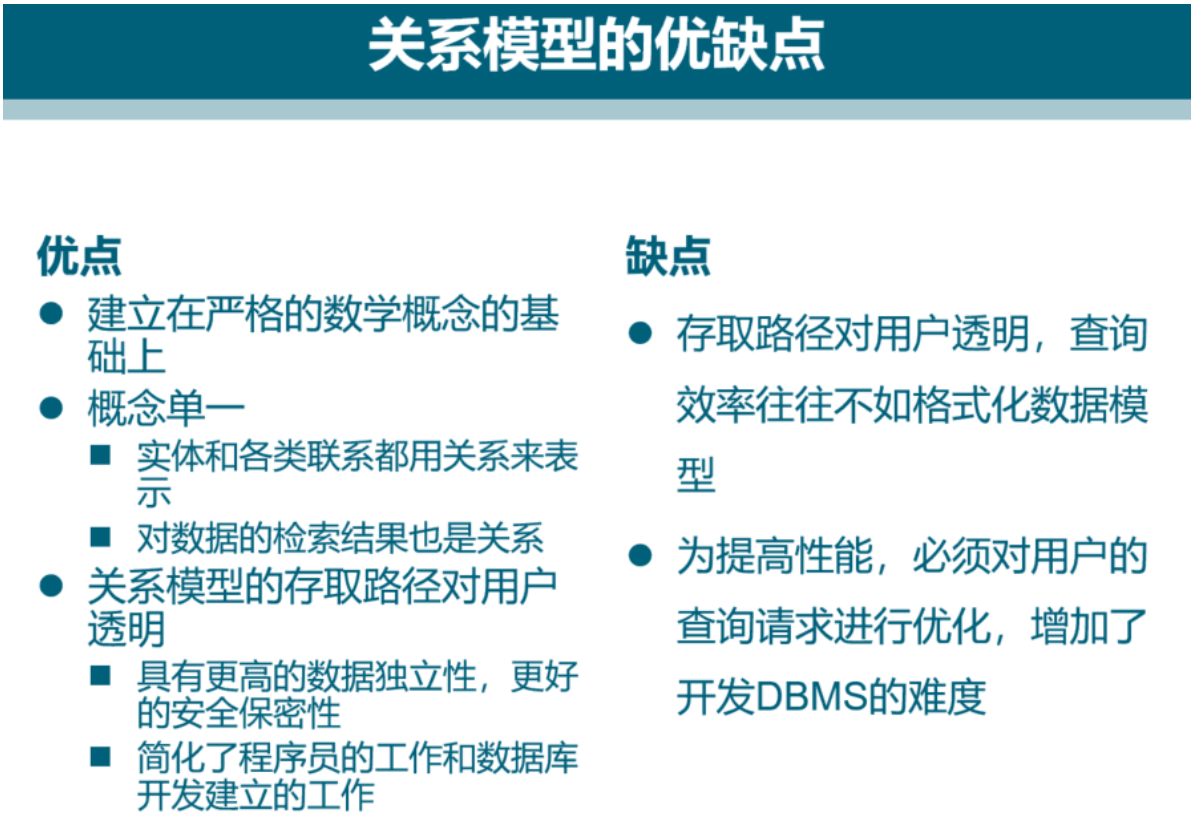
05 逻辑模型

1. 逻辑模型：按**计算机系统的观点**对数据建模，用于**DBMS**实现
    - 格式化模型（现在没人用了）
        - 层次模型（Hierarchical Model）

        - 网状模型（Network Model）
    - **关系模型**（Relational Model)）重点
        - 关系：一个表
        - 元组：一行
        - 属性：一列
        - 码：某个属性组，可以**唯一确定**一个元组
        - **域：属性的取值范围**
        - **分量：元组中的一个属性值**
        - 关系模式：对关系的描述，一般表示为关系名（属性1，属性2...）

      不允许**表中有表**!

      数据操作是**集合操作**，对象和结果都是关系（顺序无关，无相同元素）
    - 对象模型
        - 面向对象数据模型（Object Oriented Data Model）
        - 对象关系数据模型（Object Relational Data Model）
2. 关系的完整性约束条件是什么？（见09）
    1. 实体完整性
    2. 参照完整性
    3. 用户定义的完整性
3. 关系模型的优缺点?



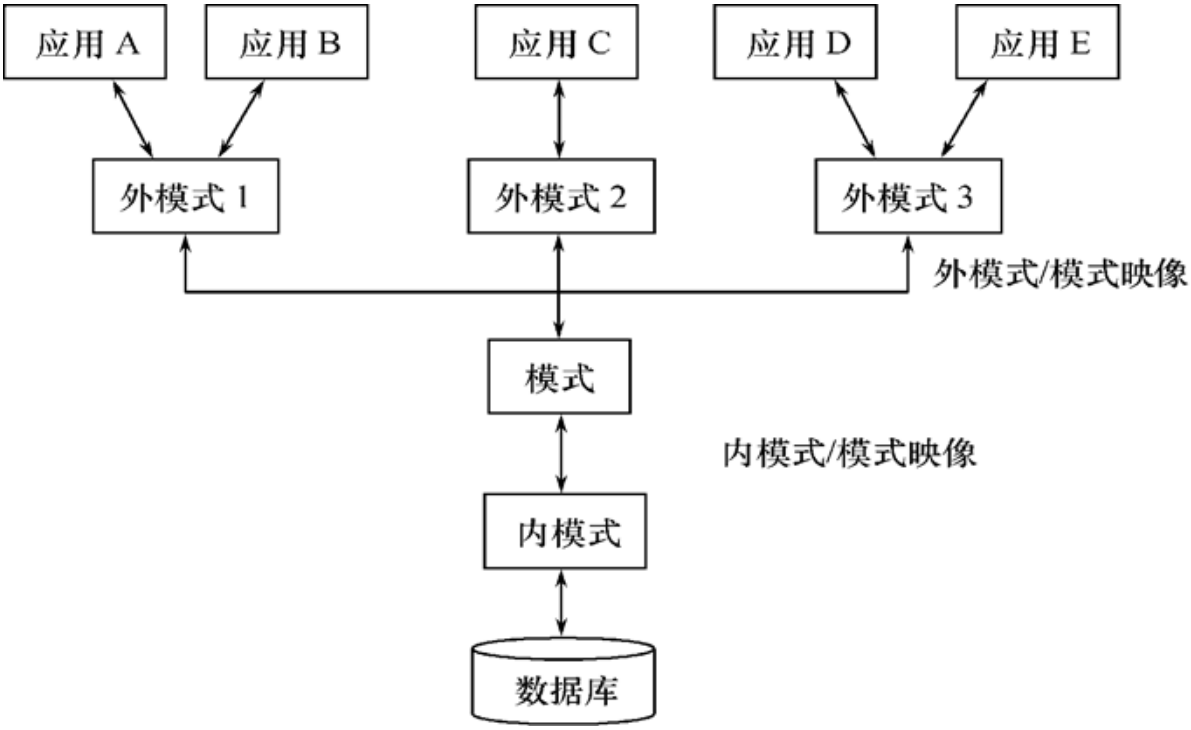
06 三级模式结构和二级映像

1. 模式和实例是什么?

- 模式：数据库**逻辑结构和特征**的描述，是**型**的描述，不涉及具体值（Class）相对稳定
- 实例：模式的一个具体值（Object）相对变动

- 例如：在学生选课数据库模式中，包含学生记录、课程记录和学生选课记录
  - 2013年的一个学生数据库实例，包含：
    - 2013年学校中所有学生的记录
    - 2013年学校开设的所有课程的记录
    - 2013年所有学生选课的记录
  - 2012年的一个学生数据库实例，包含：
    - 2012年学校中所有学生的记录
    - 2012年学校开设的所有课程的记录
    - 2012年所有学生选课的记录
  - 2012年度学生数据库模式对应的实例与2013年度学生数据库模式对应的实例是不同的

2. **三级模式结构**和**二级映像**是什么？



三级模式结构：

- 模式（**数据库的中心与关键**）：也称逻辑模式，数据库中全体数据的逻辑结构和特征的描述，所有用户的**公共数据视图**
  - 一个数据库**只有一个模式**
  - 由模式DDL定义
- 外模式：也称子模式或用户模式，数据库用户（包括应用程序员和最终用户）使用的局部数据的逻辑结构和特征的描述，数据库用户的**数据视图**，是**与某一应用有关**的数据的逻辑表示
  - 是模式的**子集**（部分镜像）
  - 一个数据库可以有**多个**外模式，一个应用程序只能使用**一个**外模式
  - 由模式DDL定义
- 内模式：也称存储模式，是数据**物理结构和存储方式**的描述，是数据在数据库内部的表示方式
  - 一个数据库只有**一个**内模式

二级映像：保证外模式的稳定性，**数据与程序**之间的独立性

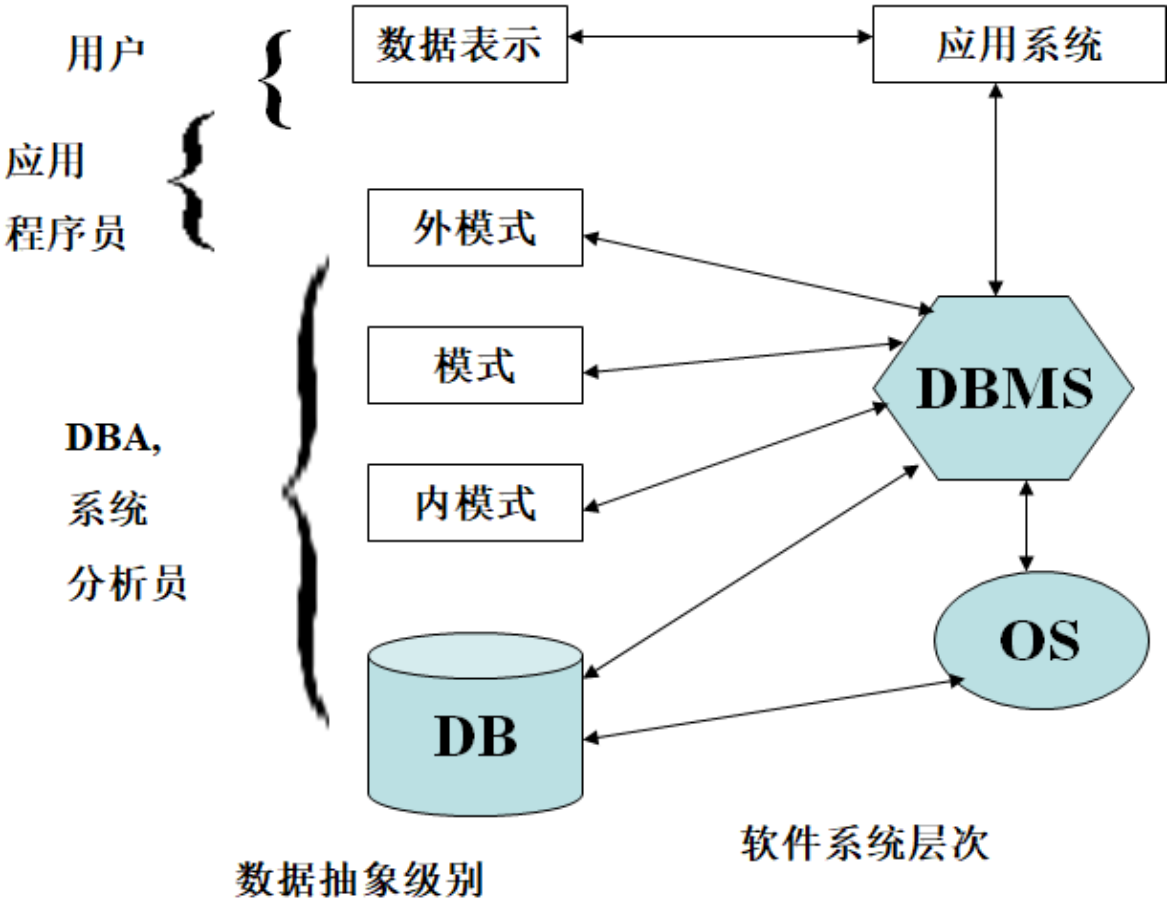
- 外模式/模式映像
  - 模式：描述的是数据的**全局逻辑结构**，外模式：描述的是数据的**局部逻辑结构**
  - 保证数据的**逻辑独立性**：模式改变时外模式可以不变，而应用程序依据外模式编写，所以保证了数据与程序的逻辑独立性
- 模式/内模式映像
  - 定义了数据**全局逻辑结构与存储结构**之间的对应关系
  - 模式 / 内模式映像是**唯一**的，通常包含在**模式**描述中
  - 保证数据的**物理独立性**：底层存储结构变，模式可不变，应用程序随之也不变

3. 为什么要有三级模式和二级映像?
   1. 实现数据与程序之间的**独立性**，使得数据的定义和描述可以从应用程序中分离出去
      - 可以讲讲逻辑独立性和物理独立性
   2. 让**数据的存取**由数据库管理系统管理，简化应用程序的维护和修改

07 数据库系统的组成

1. 数据库系统的组成是什么?

- 数据库：软硬件性能越高越好
- 数据库管理系统（及其应用开发工具）
- 应用程序
- 数据库管理员：管理两类共享资源——数据库和数据库管理系统软件



第二章 关系数据库

08 关系与关系模式

> 关系：属性名+表名+**数据**（就是一个表）
>
> 关系模式：属性名+表名
>
> **同一关系模式下可能存在多种关系**，比如随着时间的推移，表中数据发生了变化，所以关系模式是稳定的，关系是不稳定的。

1. 单一的数据结构——关系

   从集合论角度给出的关系数据结构的形式化定义

- 域：一组具有**相同数据类型**的值的集合。不同取值个数叫基数。
- 笛卡尔积：n元组的集合。笛卡尔积的基数是域的基数的连乘。
  - 例如，给出3个域：
    - D1=导师集合SUPERVISOR={张清玫，刘逸}
    - D2=专业集合SPECIALITY={计算机专业，信息专业}
    - D3=研究生集合POSTGRADUATE={李勇，刘晨，王敏}
  - D1，D2，D3的笛卡尔积为
    - D1×D2×D3 = {(张清玫，计算机专业，李勇)，(张清玫，计算机专业，刘晨)，(张清玫，计算机专业，王敏)，(张清玫，信息专业，李勇)，(张清玫，信息专业，刘晨)，(张清玫，信息专业，王敏)，(刘逸，计算机专业，李勇)，(刘逸，计算机专业，刘晨)，(刘逸，计算机专业，王敏)，(刘逸，信息专业，李勇)，(刘逸，信息专业，刘晨)，(刘逸，信息专业，王敏)}
  - 其基数为2×2×3 = 12　一共有12种组合
- 关系：笛卡尔积的子集。
  - 元组：表中的一行
  - 属性：一列
  - 候选码：某一属性组的值能**唯一标识**一个元组，里面的属性为主属性
  - 主属性：候选码中的诸属性
  - 主码：多个候选码选一个
  - 3个种类：基本关系（实际存在的二维表）、查询表和视图表

2. **关系模式（静态）**：对关系（动态）的描述。

- 关系模式可以形式化地表示为：$R（U，D，DOM，F）$
  - R：关系名
  - U：组成该关系的属性名集合
  - D：U中属性所来自的域
  - DOM：属性向域的映象集合
  - F：属性间数据的依赖关系的集合
- 关系模式通常可以简记为 $R(U)$ 或 $R(A_1，A_2，…，A_n)$
  - R：关系名
  - $A_1，A_2，…，A_n$：属性名
    - 注：域名及属性向域的映象常常直接说明为属性的类型、长度

09 关系的三类完整性约束

1. **关系的三类完整性约束**分别是什么？

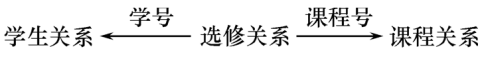
- 实体完整性（必须满足）：每个元组应该是可区分的，是**唯一**的
  - 主属性不能为空值null
- 参照完整性（必须满足）：实体及实体间的联系
  - 属性（参照关系）->外码（被参照关系，目标关系）
  - “学生”的专业号要么是空值，要么是“专业”中的某个主码值
    - **如果参照关系不是主码**，要么是null，要么是被参照关系的主码
  - 例1中，学生关系中每个元组的“专业号”属性只取两类值：
    - 空值，表示尚未给该学生分配专业
    - 非空值，这时该值必须是专业关系中某个元组的“专业号”值，表示该学生不可能分配一个不存在的专业

学号	姓名	性别	专业号	年龄
801	张三	女	01	19
802	李四	男	01	20
803	王五	男	01	20
804	赵六	女	02	20
805	钱七	男	02	19

专业号	专业名
01	信息
02	数学
03	计算机

    - 如果参照关系是主码，则不能为null，如下

- 例2中，选修关系的“学号” 与学生关系的主码“学号”相对应，选修关系的“课程号”与课程关系的主码“课程号”相对应
  - “学号”和“课程号”是选修关系的外码
  - 学生关系和课程关系均为被参照关系
  - 选修关系为参照关系



(b)

- 例2中，选修（学号，课程号，成绩）
- “学号”和“课程号”可能的取值：
  - 选修关系中的主属性，不能取空值
  - 只能取相应被参照关系中已经存在的主码值

学号	课程号	成绩
801	04	92
801	03	78
801	02	85
802	03	82
802	04	90
803	04	88

课程号	课程名	学分
01	数据库	4
02	数据结构	4
03	编译	4
04	PASCAL	2

学 号	姓 名	性 别	专 业 号	年 龄
801	张 三	女	01	19
802	李 四	男	01	20
803	王 五	男	01	20
804	赵 六	女	02	20
805	钱 七	男	02	19

- 用户定义的完整性
  - 某一**具体**关系数据库的约束条件

> 例：课程（课程号，课程名，学分）
>
> “课程名”属性必须取唯一值
>
> 非主属性“课程名”也不能取空值
>
> “学分”属性只能取值{1，2，3，4}

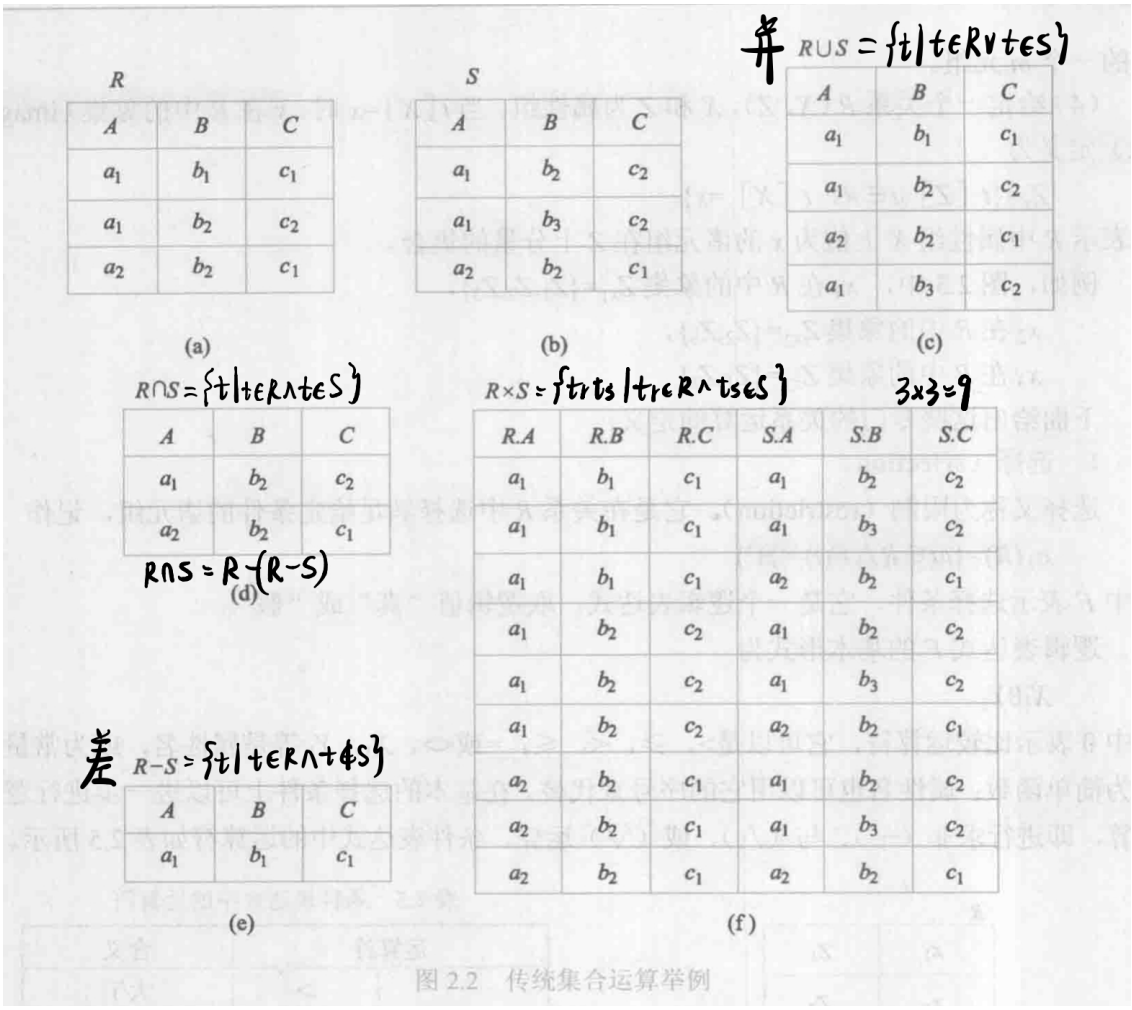
10 关系运算

关系代数运算符

运 算 符		含 义
集合运算符	∪	并
	-	差
	∩	交
	×	笛卡尔积
专门的关系运算符	σ	选择
	π	投影
	⋈	连接
	÷	除

1. 对关系的运算来表达查询，要学会怎么运算

- 传统的集合运算（二目）
  - 并
  - 差
  - 交
  - 笛卡尔积



- 专门的关系运算

  例：查询年龄小于20岁的学生。

  $\sigma_{Sage < 20}(Student)$

  结果：

  - 选择：σF(*R*) = {*t* | *t*Î*R* ∧ *F*(*t*)= '真'}

Sno	Sname	Ssex	Sage	Sdept
201215122	刘晨	女	19	IS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

  - 例：查询学生的姓名和所在系。

    即求Student关系上学生姓名和所在系两个属性上的投影

    $\pi_{Sname,Sdept}(Student)$

    结果：

  - 投影：π*A*(*R*) = { *t*[*A*] | *t* Î*R* }

Sname	Sdept
李勇	CS
刘晨	CS
王敏	MA
张立	IS

  - 连接（考自然连接）

- 

R

A	B	C
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

(a) 关系R

S

B	E
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

(b) 关系S

$R \bowtie S$ $C<E$ 常满足

$R \bowtie S$ $C<E$

A	R.B	C	S.B	E
a_1	b_1	5	b_2	7
a_1	b_1	5	b_3	10
a_1	b_2	6	b_2	7
a_1	b_2	6	b_3	10
a_2	b_3	8	b_3	10

不会除重复列

(c) 非等值连接

$R \bowtie S$ $R.B=S.B$

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

(d) 等值连接 选取B属性值相等

$R \bowtie S$

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2

特殊的等值，会除重复列

(e) 自然连接

图 2.7 连接运算举例

- 为了不丢失信息，保留悬浮元组，有时会用外连接（需要了解）



A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

(a) 外连接 保留R和S中 $R \bowtie S$ 的悬浮元组

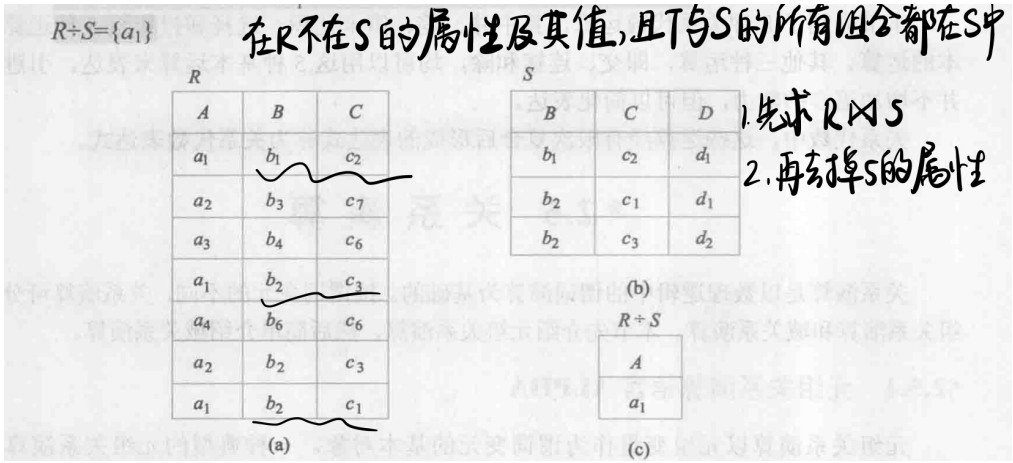
A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL

(b) 左外连接 保留左边关系R的悬浮元组 $R \bowtie S$

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
NULL	b_5	NULL	2

(c) 右外连接 保留右边关系S中的悬浮元组 $R \bowtie S$

图 2.8 外连接运算举例

- 除运算
  - 用S的属性进行筛选，但去除S的属性
  - 

$R \div S = \{a_1\}$

在R不在S的属性及其值，且T与S的所有组合都在S中

R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

(a)

S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

(b)

1. 先求 $R \bowtie S$
2. 再去掉S的属性

$R \div S$

A
a_1

(c)

> 选择对行，投影对列，连接和除运算操作多个关系

2. 综合例题

Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

(a)

Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

(b)

SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

(c)

图 2.4 学生-课程数据库

首先建立一个临时关系K:

Cno
1
3

然后求：$\pi_{Sno,Cno}(SC)\div K$

- 查询至少选修1号课程和3号课程的学生号码 。
- 查询选修了2号课程的学生的学号。
  - πSno(σCno='2'(SC))={201215121,201215122}
- 查询至少选修了一门其直接先行课为5号课程的学生姓名。

$$\pi_{Sname}(\sigma_{Cpno='5'}(Course \bowtie SC \bowtie \pi_{Sno,Sname}(Student))$$

  - 或

$$\pi_{Sname}(\pi_{Sno}(\sigma_{Cpno='5'}(Course)\bowtie SC)\bowtie \pi_{Sno,Sname}(Student))$$

- 查询选修了全部课程的学生号码和姓名。
  - $\pi_{Sno,Cno}(SC)\div\pi_{Cno}(Course)\bowtie\pi_{Sno,Sname}(Student)$

> 连接时可以先投影一下去除多余的列，当然不写也行

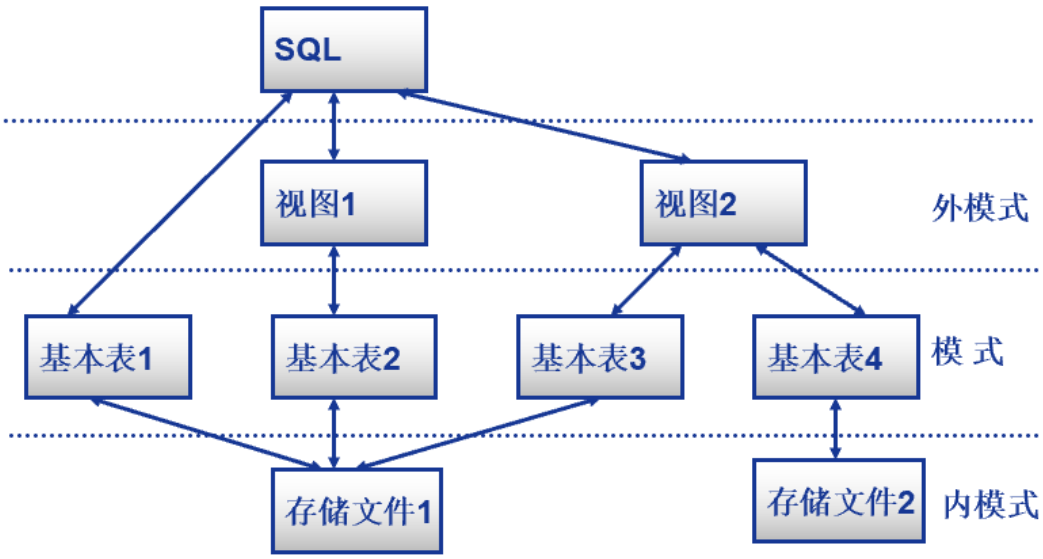
第三章

11 SQL概述

1. SQL（Structured Query Language）结构化查询语言，是关系数据库的标准语言
2. SQL的特点（考选择）
   - 综合统一
     - 集数据定义语言（DDL），数据操纵语言（DML），数据控制语言（DCL）功能于一体。
   - 高度非过程化
     - 非关系数据模型的DML“面向过程”----做什么+怎么做

        - SQL“非过程化”----做什么 存取路径的选择以及SQL的操作过程由**系统自动**完成
    - 面向**集合**的操作方式
        - 非关系数据模型面向记录
        - SQL操作元组集合
    - 以同一种语法结构提供多种使用方式
        - 可独立可嵌入（例如C，C++，Java）
    - 语言简洁，易学易用
        - 数据定义：CREATE，DROP，ALTER
        - 数据查询：SELECT
        - 数据操作：INSERT，UPDATE，DELETE
        - 数据控制：GRANT，REVOKE

3. SQL与关系数据库三级模式

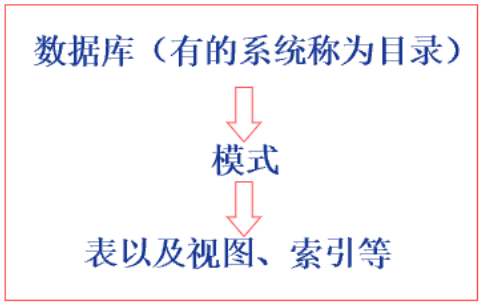


- 模式：基本表
- 内模式：存储文件
- 外模式：视图

12 数据定义

层次化的数据库对象命名机制

- 一个关系数据库管理系统的实例（Instance）中可以建立多个数据库
- 一个数据库中可以建立多个模式
- 一个模式下通常包括多个表、视图和索引等数据库对象



- 4种数据定义功能

操作对象	操作方式		
	创建	删除	修改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	ALTER INDEX

模式

- 定义
  - CREATE SCHEMA <模式名> AUTHORIZATION <用户名> [<表定义子句>| <视图定义子句>|<授权定义子句>]

> [例3.3]为用户ZHANG创建了一个模式TEST，并且在其中定义一个表TAB1
>
> CREATE SCHEMA **TEST** AUTHORIZATION **ZHANG**
>
> CREATE TABLE **TAB1** (COL1 SMALLINT,
>
> COL2 INT,
>
> COL3 CHAR(20),
>
> COL4 NUMERIC(10,3),
>
> COL5 DECIMAL(5,2));

- 删除
  - DROP SCHEMA <模式名> <CASCADE|RESTRICT>

- CASCADE（**级联**）

  删除模式的同时把该模式中所有的数据库对象全部删除

  RESTRICT（限制）

  如果该模式中定义了下属的数据库对象（如表、视图等），则拒绝该删除语句的执行。

> [例3.4] DROP SCHEMA ZHANG CASCADE;

基本表

- 定义
  - CREATE TABLE <表名>

    (<列名> <数据类型>[<列级完整性约束条件>]

    [,<列名> <数据类型>[<列级完整性约束条件>]]

    …

    [,<表级完整性约束条件>]);
  - 列级完整性：PRIMARY KEY UNIQUE NOT NULL等
  - 表级完整性：
    - PRIMARY KEY(Sno,Cno)
    - FOREIGN KEY (Cno) REFERENCES Course(Cno)等
- 修改
  - ALTER TABLE <表名>

    [ADD[COLUMN] <新列名> <数据类型> [完整性约束]]

    [ADD <表级完整性约束>]

    [DROP [COLUMN] <列名> [CASCADE| RESTRICT]]

    [DROP CONSTRAINT<完整性约束名>[RESTRICT | CASCADE]]

    [ALTER COLUMN <列名><数据类型>] ;
- 删除
  - DROP TABLE <表名> [RESTRICT| CASCADE] ;
  - RESTRICT：删除表是有限制的。

    如果存在依赖该表的对象，则此表不能被删除

    CASCADE：删除该表没有限制。

    在删除基本表的同时，相关的依赖对象一起删除

索引（x)

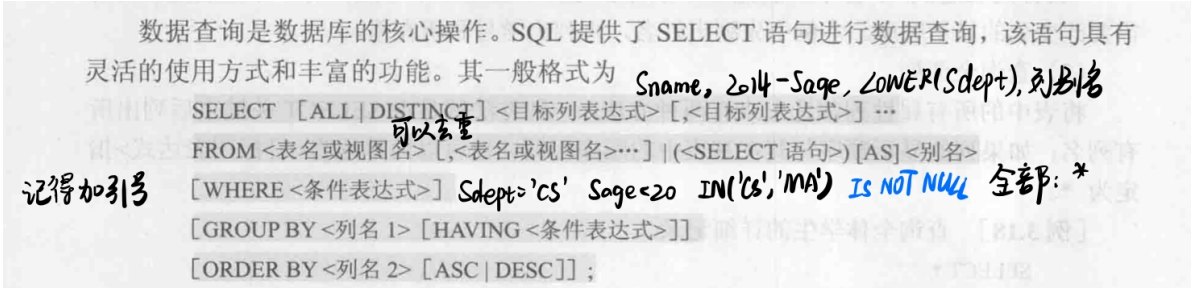
- 执行查询时自动选择合适的索引作为存取路径
- 建立
  - CREATE [UNIQUE] [CLUSTER] INDEX <索引名>

    ON <表名>(<列名>[<次序>][,<列名>[<次序>]]…);

    > CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);
- 修改（名字）
  - ALTER INDEX <旧索引名> RENAME TO <新索引名>
- 删除

- DROP INDEX <索引名>;（MySQL中需要指明表名 +ON <表名>）

13 单表查询



- 可以是Sno(列)/*(与原表顺序相同)/2014-Sage(表达式)/别名(改变列标题)
- DISTINCT：去掉重复行
- WHERE表示指定条件（作用于表）
  - BETWEEN AND(范围) IN(集合)LIKE(匹配字符)IS NULL(空值)AND OR(多重条件查询)
  - ORDER BY(升降排列)
- 聚集函数 COUNT SUM AVG MAX MIN(需要指定DISTINCT或ALL)
  - [例3.41] 查询学生总人数。

    ```
    SELECT COUNT(*)
    FROM Student;
    ```

  - [例3.42] 查询选修了课程的学生人数。（会清除Sno is NULL的）

    ```
    SELECT COUNT(DISTINCT Sno)
    FROM SC;
    ```

  - [例3.43] 计算1号课程的学生平均成绩。

    ```
    SELECT AVG(Grade)
    FROM SC
    WHERE Cno= ' 1 ';
    ```

  - [例3.44] 查询选修1号课程的学生最高分数。

    ```
    SELECT MAX(Grade)
    FROM SC
    WHERE Cno='1';
    ```

  - [例3.45] 查询学生201215012选修课程的总学分数。

    ```
    SELECT SUM(Ccredit)
    FROM SC,Course
    WHERE Sno='201215012' AND SC.Cno=Course.Cno;
    ```

  - WHERE不能用聚集函数作为条件表达式，聚集函数只用于SELECT和GROUP BY中的HAVING子句。
- GROUP BY按照某一列或多列的值分组（作用于组）
  - 如所有相同Cno值的元组为一组，然后**对每一组作用聚集函数**进行计算。若分组后再进行筛选，则用HAVING指定条件。
  -

- [例3.46] 求各个课程号及相应的选课人数。

  SELECT Cno，COUNT(Sno)

  FROM SC

  GROUP BY Cno;

- 查询结果可能为：

Cno	COUNT(Sno)
1	22
2	34
3	44
4	33
5	48

- [例3.47] 查询选修了3门以上课程的学生学号。

  SELECT Sno

  FROM SC

  GROUP BY Sno

  HAVING COUNT(*) >3;

- HAVING短语与WHERE子句的区别：
  - 作用对象不同
  - WHERE子句作用于基表或视图，从中选择满足条件的元组
  - HAVING短语作用于组，从中选择满足条件的组。

- [例3.48]查询平均成绩大于等于90分的学生学号和平均成绩
- 因为WHERE子句中是不能用聚集函数作为条件表达式，下面的语句是不对的：

  SELECT Sno, AVG(Grade)

  FROM SC

  WHERE AVG(Grade)>=90（WHERE：只有一个元组 不对）

  GROUP BY Sno;

- 正确的查询语句应该是：

  SELECT Sno, AVG(Grade)

  FROM SC

  GROUP BY Sno

  HAVING AVG(Grade)>=90;

- HAVING写在GROUP BY后面

14 连接查询

嵌套循环法、排序合并法、索引连接

- 等值连接与非等值连接

- 等值连接：连接运算符为=
- [例 3.49] 查询每个学生及其选修课程的情况

```
SELECT  Student.*, SC.*
FROM    Student, SC
WHERE  Student.Sno = SC.Sno;
```

- [例 3.50] 对[例 3.49]用自然连接完成。

```
SELECT Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade（合并了）
FROM    Student,SC
WHERE  Student.Sno = SC.Sno;
```

Student.Sno	Sname	Ssex	Sage	Sdept	SC.Sno	Cno	Grade
201215121	李勇	男	20	CS	201215121	1	92
201215121	李勇	男	20	CS	201215121	2	85
201215121	李勇	男	20	CS	201215121	3	88
201215122	刘晨	女	19	CS	201215122	2	90
201215122	刘晨	女	19	CS	201215122	3	80

- 自身连接：取两个别名，可选择间接先修课等
  - [例 3.52]查询每一门课的间接先修课（即先修课的先修课）

```
SELECT  FIRST.Cno, SECOND.Cpno
FROM  Course  FIRST, Course  SECOND
WHERE FIRST.Cpno = SECOND.Cno;
```

- 多表连接：两个以上的表进行连接
  - [例3.54]查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno, Sname, Cname, Grade
FROM    Student, SC, Course    /*多表连接*/
WHERE Student.Sno = SC.Sno
            AND SC.Cno = Course.Cno;
```

15 嵌套查询

基本可以转换为自身连接，多表连接

- 注意子查询不能有ORDER BY，ORDER BY只能对最终查询结果排序
  - 不相关子查询：由里向外逐层处理。
  - 相关子查询：从外到里，重复每一层
- 谓词IN

- 谓词EXISTS
  - 只返回逻辑真假，若为真，就把SELECT的东西送入结果表
  - 不可被其他子查询等价替换，但可替换IN/比较/ANY ALL等子查询
    - [例 3.61] 查询没有选修1号课程的学生姓名。

```
SELECT Sname
FROM   Student
WHERE NOT EXISTS
        (SELECT *
         FROM SC
         WHERE Sno = Student.Sno AND Cno='1');
```

- 谓词ANY（SOME）或ALL
  - [例 3.59] 查询非计算机科学系中比计算机科学系所有学生年龄都小的学生姓名及年龄。
    - 方法一：用ALL谓词

```
SELECT Sname,Sage
FROM Student
WHERE Sage < ALL
        (SELECT Sage
        FROM Student
        WHERE Sdept= ' CS ')
 AND Sdept <> ' CS ';
```

    - 方法二：用聚集函数

```
SELECT Sname,Sage
FROM Student
WHERE Sage <
    (SELECT MIN(Sage)
    FROM Student
    WHERE Sdept= ' CS ')
AND Sdept <>' CS ';
```

  和多值进行了比较（聚集函数效率更高）

16 集合查询

- UNION并 INTERSECT交 EXCEPT差
  - UNION返回若干个查询结果，但重复时不显示
  - UNION ALL显示重复的

- [例 3.68] 查询计算机科学系的学生与年龄不大于19岁的学生的差集。

```
SELECT *
FROM Student
WHERE Sdept='CS'
EXCEPT
SELECT *
FROM Student
WHERE Sage <=19;
```

- [例3.68]实际上是查询计算机科学系中年龄大于19岁的学生

```
SELECT *
FROM Student
WHERE Sdept= 'CS' AND
Sage>19;
```

18 插入元组

- [例3.69]将一个新学生元组（学号：201215128;姓名：陈冬;性别：男;所在系：IS;年龄：18岁）插入到Student表中。（指定了属性列）

```
INSERT
INTO Student
(Sno,Sname,Ssex,Sdept,Sage)
VALUES ('201215128','陈冬','男','IS',18);
```

- [例3.70]将学生张成民的信息插入到Student表中。（没指定列）

```
INSERT
INTO Student
VALUES ('201215126','张成民','
男',18,'CS');
```

- [例3.71] 插入一条选课记录（ '200215128','1 '）。

```
INSERT
INTO SC(Sno,Cno)
VALUES ('201215128 ',' 1 ');
```

  - 关系数据库管理系统将在新插入记录的Grade列上自动地赋空值。
  - 或者：

```
INSERT
INTO SC
VALUES (' 201215128 ',' 1 ',NULL);
```

19 修改元组

- [例3.75] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC
SET    Grade=0
WHERE Sno  IN
    (SELETE Sno
     FROM    Student
     WHERE  Sdept= 'CS' );
```

20 删除元组

- [例3.76] 删除学号为201215128的学生记录。

```
DELETE
FROM Student
WHERE Sno= 201215128 ';
```

21 空值

- 产生：没赋值/修改/外连接/关系运算
- 判断：IS NULL/**IS NOT NULL** （注意IS一直有）
- 约束条件：NOT NULL/UNIQUE/PRIMARY KEY不能取空值
- 逻辑运算：TRUE FALSE UNKNOWN

- [例 3.83] 选出选修1号课程的不及格的学生以及缺考的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='1'
UNION
SELECT Sno
FROM SC
WHERE Grade IS NULL AND Cno='1'
```

- 或者

```
SELECT Sno
FROM SC
WHERE Cno='1' AND (Grade<60 OR Grade IS NULL);
```

注意任何情况下都要考虑空值的情况，比如**缺考的学生不属于Grade<60**，要专门写Grade IS NULL

表3.8 逻辑运算符真值表

x	y	x AND y	x OR y	NOT x
T	T	T	T	F
T	U	U	T	F
T	F	F	T	F
U	T	U	T	U
U	U	U	U	U
U	F	F	U	U
F	T	F	T	T
F	U	F	U	T
F	F	F	F	T

T表示TRUE，F表示FALSE，U表示UNKNOWN

22 视图

1. 基本概念

- 建立
  - CREATE VIEW

    <视图名> [(<列名> [,<列名>]...)]

    AS <子查询>

    [WITH CHECK OPTION];

- WITH CHECK OPTION：增删改时要保证更新、插入或删除的行满足视图定义的**谓词条件**（即**子查询**中的条件表达式）
  - 如create view IS_student
    as
    select sno,sname,sage
    from student
    where sdept='IS'
    with check option;
    加上了with check option后，插入修改删除时DBAS会自动加上Sdept = 'IS'操作。
- **行列子集视图**：从单个表导出某些列，保留主码
- 实现增删改视图-->增删改基本表：不要写聚集函数
- 指定属性列比较好，否则基表结构改变后视图不能正常工作

- 删除
  - DROP VIEW <视图名>[CASCADE];
    - 删除基表后视图无法使用，但定义需要显式删除。
- 查询：视图消解
  - 对行列子集视图可以正确转换，遇到非行列子集视图则不一定。如该写HAVING时写了WHERE。
    - 视图消解法的局限
      - 有些情况下，视图消解法不能生成正确的查询。
    - [例3.94]在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩

```
SELECT *
FROM   S_G
WHERE  Gavg>=90;
```

    - S_G视图的子查询定义：

```
CREATE VIEW S_G (Sno,Gavg)
AS
SELECT  Sno,AVG(Grade)
FROM  SC
GROUP BY Sno;
```

    - 错误：

```
SELECT Sno，AVG(Grade)
FROM    SC
WHERE  AVG(Grade)>=90
GROUP BY Sno;
```

    - 正确：

```
SELECT  Sno,AVG(Grade)
FROM  SC
GROUP BY Sno
HAVING AVG(Grade)>=90;
```

      - 或者

```
SELECT *
FROM  (SELECT Sno,AVG(Grade)
            FROM  SC
            GROUP BY Sno) AS S_G(Sno,Gavg)
  WHERE Gavg>=90;
```

- 更新
  - **肯定是同步的**，毕竟是虚表。
  - 一般来说，行列子集视图可更新，其他（如AVG等）不能更新。（可看书本规定）

2. 视图有哪些作用？使用视图的理由？
   - 简化用户的操作
   - 使用户能以多种角度看待同一数据
   - 视图对重构数据库提供了一定程度的逻辑独立性
     - 外模式不变，即使模式变更也不影响
   - 对机密数据提供安全保护
     - 每个用户看到的视图不一样
   - 可以更清晰的表达查询
3. 哪些事情视图做不到？

- **行列子集视图**是可更新的，但部分特殊视图不允许更新。
- 视图消解法有局限性，有时不能正确地查询。

第四章

23 安全性概述

数据库的安全性是指保护数据库以防止不合法使用所造成的**数据泄露、更改或破坏** 。

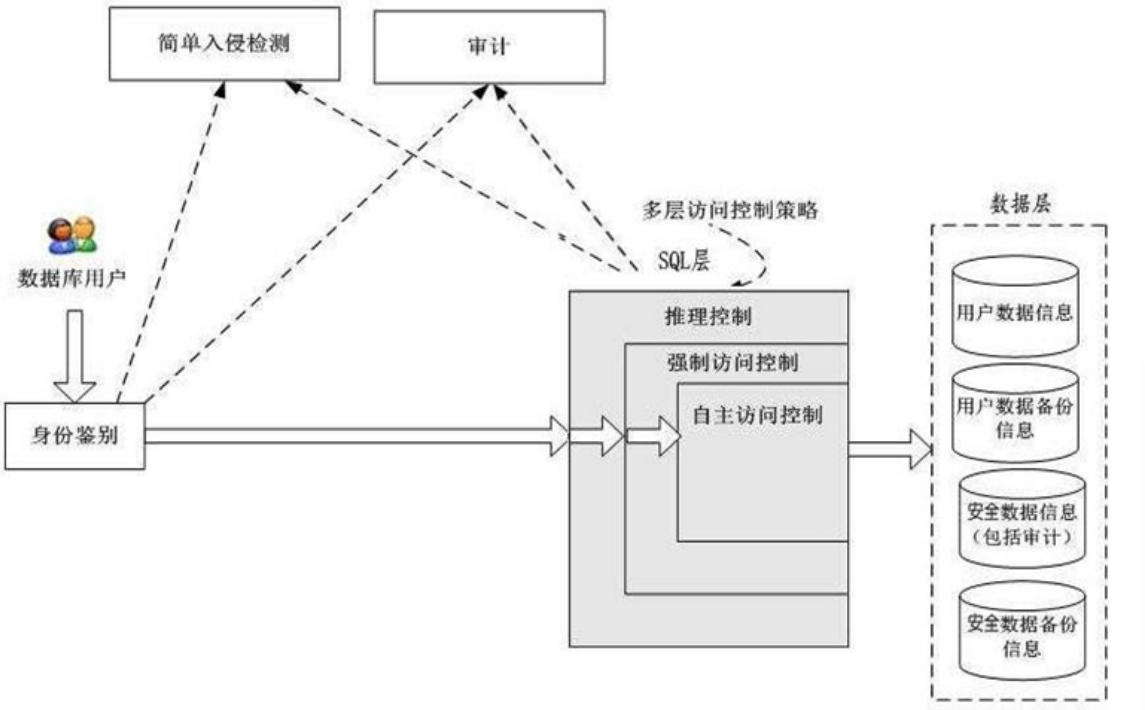
系统安全保护措施是否有效是数据库系统主要的性能指标之一。

不安全因素：

- 非授权用户对数据库的恶意存取和破坏
- 数据库中重要或敏感的数据被泄露
- 安全环境的脆弱性

安全标准简介：

- TCSEC/TDI
  - C1：自主存取控制（DAC）
  - B1：强制存取控制（MAC）以及审计
  - B1以上为安全产品
- CC
  - 安全功能要求和安全保证要求



24 存取控制

1. 存取机制的组成是什么？

- 定义用户权限
  - 定义后存储在数据字典中

- 合法权限检查
  - 查找数据字典，看请求是否超出权限
- **用户权限定义**和**合法权检查机制**一起组成了DBMS的存取控制子系统

自主存取控制（DAC）

- 用户权限组成
  - 数据库对象
  - 操作类型
- 定义存取权限称为授权

对象类型	对象	操作类型
数据库模式	模式	CREATE SCHEMA
	基本表	CREATE TABLE，ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT，INSERT，UPDATE，DELETE，REFERENCES，ALL PRIVILEGES
	属性列	SELECT，INSERT，UPDATE，REFERENCES，ALL PRIVILEGES

给列UPDATE要遵循主码和其他约束

给列INSERT要给主码的INSERT权限

- GRANT
  - GRANT <权限>[,<权限>]…

    ON <对象类型> <对象名>[,<对象类型> <对象名>]…

    TO <用户>[,<用户>]…

    [WITH GRANT OPTION];//若指定：可以再授予
  - 不允许循环授权！
- REVOKE
  - REVOKE <权限>[,<权限>]…

    ON <对象类型> <对象名>[,<对象类型><对象名>]…

    FROM <用户>[,<用户>]…[CASCADE | RESTRICT];
- CREATE USER创建用户时实现对象的授权
  - CREATE USER <username>

    [WITH][DBA|RESOURCE|CONNECT];

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库，执行数据查询和操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	可以	可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限

- 角色：被命名的权限，权限的集合
  - 创建：CREATE ROLE <角色名>
  - 给角色授权：GRANT <权限>[,<权限>]...

    ON <对象类型>对象名

    TO <角色>[,<角色>]...
  - 角色授予另一个：GRANT <角色1>[,<角色2>]...

    TO <角色3>[,<用户1>]...

    [WITH ADMIN OPTION]
  - 收回权限：REVOKE <权限>[,<权限>]...

    ON <对象类型> <对象名>

    FROM <角色>[,<角色>]...

  - [例4.11] 通过角色来实现将一组权限授予一个用户。步骤如下：
    - 首先创建一个角色 R1

      ```
      CREATE ROLE R1;
      ```
    - 然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限

      ```
      GRANT SELECT, UPDATE, INSERT
      ON TABLE Student
      TO R1;
      ```
    - 将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限

      ```
      GRANT R1
      TO 王平,张明,赵玲;
      ```
    - 可以一次性通过R1来回收王平的这3个权限

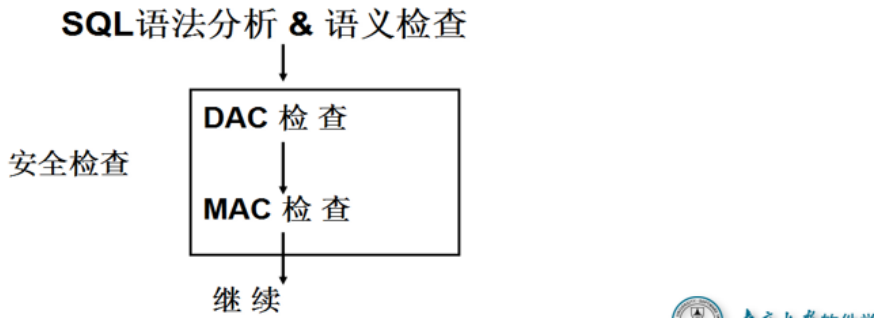
      ```
      REVOKE R1
      FROM 王平;
      ```
- 缺点：存在“**无意泄露**”，可以传递副本
  - 虽然甲要求乙不能传播，但乙可以将数据备份，获得副本进行传播

强制存取控制（MAC）

- 主体：用户或进程
  - 敏感度标记：许可证级别
- 客体：文件、基本表、索引、视图等
  - 敏感度标记：密级

- 每个实例指派一个敏感度标记，通过对比主客体标记，确定主体是否能存取客体
  - 主体的许可证级别>=客体的密级----->可读取
  - 主体的许可证级别<=客体的密级----->可写：用户可以对写入的对象赋予**高于自己的密级**，否则密级可以高流向低，造成数据泄露
- 标记和数据不可分
- 要首先实现自主存取控制，进行DAC+MAC两次安全检查

- 自主存取控制与强制存取控制共同构成数据库管理系统的安全机制，先进行自主存取控制检查，通过自主存取控制检查的数据对象再由系统进行强制存取控制检查，只有通过强制存取控制检查的数据对象方可存取。



25 视图机制、审计、数据加密

1. 视图机制：通过视图进行**数据隐藏**

- 把要保密的数据对无权存取这些数据的用户隐藏起来->对数据已经进行了一次选择

2. 审计：启用一个专用的**审计日志**（Audit Log）将用户对数据库的所有操作记录在上面

- 可选性：因为费时间和空间
- AUDIT语句：设置审计功能
- NOAUDIT语句：取消审计功能
- 用户级审计、系统级审计
- 存在数据字典中，提供事后检查的安全机制

3. 数据加密

- 明文->密文
- 存储加密
  - 透明
  - 非透明
- 传输加密
  - 链路加密
    - 报头和报文**均**加密
  - 端到端加密
    - 发送端加密，接收端解密
    - 只加密**报文**不加密报头

4. 其他安全性保护

- 推理控制：避免用已知数据推得更高级数据
- 隐蔽信道：间接信息传递

- 数据隐私：已掌握的信息可以推理出不愿他人知道或他人不便知道的数据，如确定身高、性别、住址后只有一个人

第五章

26 数据库完整性

1. 数据库完整性的组成是什么?

- 数据的正确性：符合语义，**实际**情况
- 数据的相容性：符合**逻辑**，防止恶意非法操作

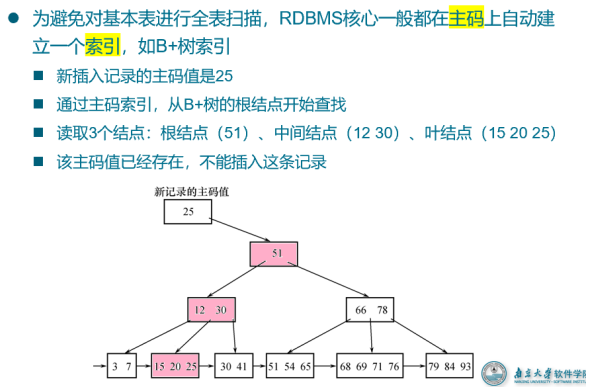
2. 数据库管理系统怎么实现完整性?

- 提供定义完整性约束条件的机制
- 提供完整性**检查**的方法
- 违约处理

27 实体完整性

1. 保证实体完整性的方式：定义主码

- 是否唯一
  - 全表扫描防止主码重复（效率低）
  - **B+树**索引（高效）



- 是否为空，只要有一个为空就拒绝插入或修改

28 参照完整性

1. 保证参照完整性的方式：定义外码及外码参照的表的**主码**
2. 破坏参照完整性时：
   - 拒绝：默认策略，不允许该操作的执行
   - 级联：删除或修改参照表中的所有造成不一致的元组
   - 设置为空值
     - 外码可以为空，但只能在参照表里为空！否则被参照表不满足实体完整性！

- 显式说明参照完整性的违约处理示例：
  - [例5.4] 显式说明参照完整性的违约处理示例

```
CREATE TABLE SC
( Sno   CHAR(9)  NOT NULL,
  Cno   CHAR(4)  NOT NULL,
  Grade  SMALLINT,
  PRIMARY KEY(Sno,Cno),
  FOREIGN KEY (Sno) REFERENCES Student(Sno)
      ON DELETE CASCADE       /*级联删除SC表中相应的元组*/
      ON UPDATE CASCADE,      /*级联更新SC表中相应的元组*/
  FOREIGN KEY (Cno) REFERENCES Course(Cno)
      ON DELETE NO ACTION
     /*当删除course 表中的元组造成了与SC表不一致时拒绝删除*/
      ON UPDATE CASCADE
       /*当更新course表中的cno时，级联更新SC表中相应的元组*/
);
```

29 用户定义完整性

1. 保证参照完整性的方式：

- 属性上约束（针对**一个**属性）
  - 列值非空NOT NULL
  - 列值唯一UNIQUE
  - 检查列值是否满足一个条件表达式CHECK
    - `Ssex CHAR(2) CHECK (Ssex IN ('男','女'))`
    - `Grade SMALLINT CHECK (Grade>=0 AND Grade<=100)`
- 元组上约束（针对**多个**属性）
  - 可以设置**不同属性**之间取值的相互约束状态
    - `CHECK (Ssex='女' OR Sname NOT LIKE 'Ms.%')`

2. 完整性约束命名子句

- CONSTRAINT <完整性约束条件名><完整性约束条件>
- 使用ALTER TABLE语句修改表中的完整性限制

31 触发器

- 创建：CREATE TRIGGER <触发器名>

  {BEFORE | AFTER} <触发事件> ON <表名>

  REFERENCING NEW|OLD ROW AS<变量>

  FOR EACH {ROW | STATEMENT}

  [WHEN <触发条件>]<触发动作体>
  - 行级每行都执行，语句级只执行一次
  - NEW OLD表示操作前后的新值和旧值
- 触发：事件激活，自动执行 “谁先创建谁先执行”
- 删除：DROP TRIGGER <触发器名> ON <表名>;

- [例5.22] 将每次对表Student的插入操作所增加的学生个数记录到表StudentInsertLog中。

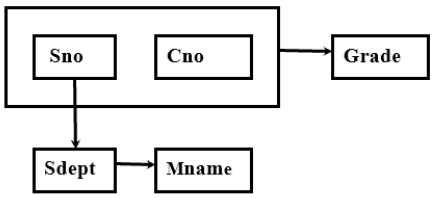
```
CREATE TRIGGER Student_Count
AFTER INSERT ON Student
    /*指明触发器激活的时间是在执行INSERT后*/
REFERENCING
NEW TABLE AS DELTA
FOR EACH STATEMENT
/*语句级触发器，即执行完INSERT语句后下面的触发动作体才执行一次*/
        INSERT INTO StudentInsertLog (Numbers)
        SELECT COUNT(*) FROM DELTA
```

第六章

32 关系模式与范式

1. 关系模式是什么?
   - 可以把关系模式看作一个三元组：*R*<*U*,*F*>
   - U是一组属性，F是属性组U上的一组**数据依赖**，如学号可以唯一确定学生的名字。
   - 当且仅当U上的一个关系r满足F时，称为r是关系模式*R*<*U*,*F*>的一个关系
2. 什么是第一范式?
   - 每个分量必须是**不可分开**的数据项。
   - 满足这个条件的关系模式属于1NF。
3. 数据依赖是什么?
   - 关系内部**属性与属性之间**的一种约束关系
   - 函数依赖：Sname=*f*(Sno)，Sdept=*f*(Sno) 学号可以决定学生的姓名和院系，记作Sno→Sname，Sno→Sdept

- [例6.1] 建立一个描述学校教务的数据库。涉及的对象包括：
  - 学生的学号（Sno）
  - 所在系（Sdept）
  - 系主任姓名（Mname）
  - 课程号（Cno）
  - 成绩（Grade）



- 假设学校教务的数据库模式用一个单一的关系模式Student来表示，则该关系模式的属性集合为：
  - *U* ={Sno, Sdept, Mname, Cno, Grade}
- 现实世界的已知事实（语义）：
  - 一个系有若干学生，但一个学生只属于一个系；
  - 一个系只有一名（正职）负责人；
  - 一个学生可以选修多门课程，每门课程有若干学生选修；
  - 每个学生学习每一门课程有一个成绩。
- 由此可得到属性组U上的一组函数依赖F：
  - *F*={Sno→Sdept, Sdept→ Mname, (Sno, Cno)→ Grade}

   - 多值依赖

4. 1NF存在的问题？（4）

- 关系模式Student<U, F>中存在的问题：
  - 数据冗余
    - 浪费大量的存储空间
      - 每一个系主任的姓名重复出现，重复次数与该系所有学生的所有课程成绩出现次数相同。
  - 更新异常（Update Anomalies）
    - 数据冗余，更新数据时，维护数据完整性代价大。
      - 某系更换系主任后，必须修改与该系学生有关的每一个元组。
  - 插入异常（Insertion Anomalies）
    - 如果一个系刚成立，尚无学生，则无法把这个系及其系主任的信息存入数据库。
  - 删除异常（Deletion Anomalies）
    - 如果某个系的学生全部毕业了，则在删除该系学生信息的同时，把这个系及其系主任的信息也丢掉了。

- 解决：用**规范化理论**改造关系模式，拆成三个。

33 函数依赖与码

1. 什么是函数依赖？
   1. 能否通过X确定Y
   2. 语义范畴的概念
2. 函数依赖的几组对比分类？
   1. 平凡VS非平凡：平凡永远成立，所以关注非平凡。

- 在关系模式R(U)中，对于U的子集X和Y，
  - 如果X→Y，但$Y \not\subseteq X$，则称X→Y是非平凡的函数依赖
  - 若X→Y，但$Y \subseteq X$，则称X→Y是平凡的函数依赖
- 例：在关系SC(Sno, Cno, Grade)中，

  非平凡函数依赖：(Sno, Cno) → Grade

  平凡函数依赖：(Sno, Cno) → Sno

  (Sno, Cno) → Cno
  - 对于任一关系模式，平凡函数依赖都是必然成立的，它不反映新的语义，因此若不特别声明，我们总是讨论非平凡函数依赖。

   2. 完全VS部分：完全是少一个属性都推不出来，部分是有多的属性，如Cno可以去掉

- 定义6.2 在$R(U)$中，如果$X \to Y$，并且对于X的任何一个真子集X'，都有 $X' \not\to Y$，则称Y对X完全函数依赖，记作 $X \xrightarrow{F} Y$。
- 若$X \to Y$，但Y不完全函数依赖于X，则称Y对X部分函数依赖，记作$X \xrightarrow{P} Y$
- 由于：Sno $\not\to$Grade，Cno $\not\to$ Grade，Sno $\to$ Sdept

  因此：(Sno, Cno) $\xrightarrow{F}$ Grade

  (Sno, Cno) $\xrightarrow{P}$ Sdept

  (Sno, Cno) $\xrightarrow{P}$ Sdept

3. 传递：通过两个非平凡的X->Y,Y->Z,得到X-传递->Z

- 定义6.3 在$R(U)$中，如果$X \to Y (Y \not\subseteq X)$，$Y \not\to X$，$Y \to Z$，$Z \not\subseteq Y$，则称Z对X传递函数依赖(transitive functional dependency)。
- 记为：$X \xrightarrow{传递} Z$。
  - 注: 如果$Y \to X$，即$X \longleftrightarrow Y$，则Z直接依赖于X，而不是传递函数依赖。
  - [例] 在关系Std(Sno, Sdept, Mname)中，有：
    - Sno → Sdept，Sdept → Mname，
    - Mname传递函数依赖于Sno　　Sno-传递->Mname

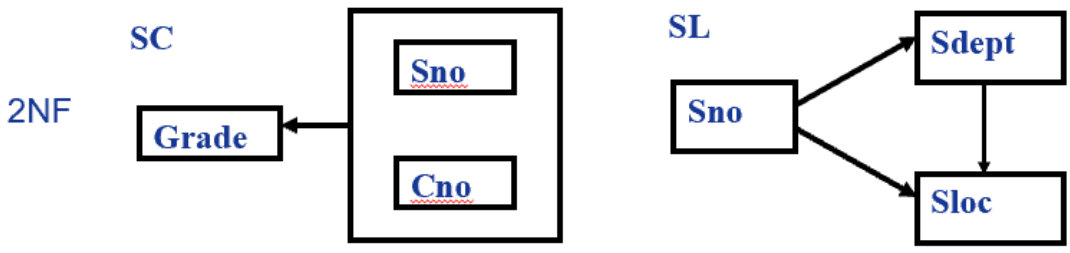
> 个人观点：我觉得传递依赖和直接函数依赖的区别**在于语义上的区别**：用一个例子来说明：中考结束了，高一的一个学生对应一个班级，一个班级对应一个班主任；我们将**学生分到一个班级**和**给班级分一个班主任是可以分开操作的（这句话好好想一想）**，而分班主任完全可能在学生录取到某个学校之前就确定好了，比如我周老师分配到1班。虽然最后的结果是一个学生必然对应一个班主任。。
>
> 我太懒了不给学生上课，虽然学生都很高兴，但是校长决定把我给炒了，然后换个认真的班主任--李萌萌，如果把（学生，班级，班主任）放到一个关系里存，那我们必须得更改所有的1班的班主任的姓名，但是如果我们分开来，（学生，班级），（班级，班主任）那么我们只需要修改（班级，班主任）里的一个元组就行了。虽然逻辑上学生确实对应一个班主任，但是如果我们建立（学生，班主任）这个关系，我们**修改的数量和1班学生的个数是一样的**。。。。
>
> 希望这个例子能很好地帮助你理解两个概念的区别。
>
> 另外直接依赖和传递依赖到底是不是一回事 这个链接里面也提出了这个问题，然后二楼的回答我觉得挺好的，对于应试做题来说，题目给出来的就是直接函数依赖，自己推导出来的就是传递依赖。

3. 什么是码？在多个候选码里面选一个当主码。
   -

- 定义6.4 设K为$R<U,F>$中的属性或属性组合。若$K \xrightarrow{F} U$，则K称为R的一个候选码(Candidate Key)。//完全函数依赖
  - 主属性：包含在任意一个**候选码**中的属性
  - 全码：整个属性组都是码
  - 外码：不是这个模式的码，是另一个模式的码
    - SC(Sno,Cno,Grade)中，Sno不是码，Sno是S(Sno,Sdept,Sage)的码，则Sno是SC的外码
      - 可表示关系间的联系

34 1NF,2NF,3NF

1. 1NF：R的所有属性都是**不可分**的基本数据项
2. 2NF：R∈1NF，并且每一个**非主属性**都完全函数依赖于任何一个候选码
   - 例子中有两类非主属性：
     - 一类如Grade，它对码完全函数依赖
     - 另一类如Sdept、Sloc，它们对码不是完全函数依赖
   - 解决方法：
     - 用投影分解把关系模式S-L-C分解成两个关系模式
       - SC(Sno,Cno,Grade)
       - S-L(Sno,Sdept,Sloc)



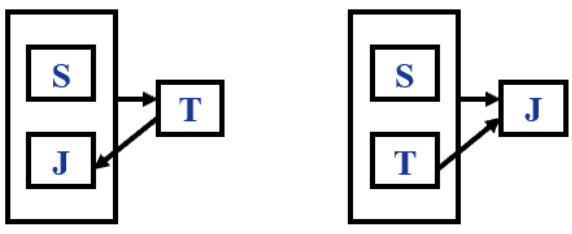
3. 3NF：每个非主属性既不**传递依赖**于码，也不部分依赖于码。（SL有从传递依赖）
   - SC没有传递依赖，因此SC ∈ 3NF
   - S-L中Sno →Sdept(Sdept ↛ Sno), Sdept→Sloc，可得$Sno \xrightarrow{传递} Sloc$。
   - 解决的办法是将S-L分解成
     - S-D(Sno,Sdept)∈ 3NF
     - D-L(Sdept,Sloc)∈ 3NF

1NF	2NF	3NF	BCNF
数据冗余	变少了		
插入异常	有改进		消除
删除异常	大问题	有改进	消除
更新异常	有改进		

35 BCNF

1. 定义：若X->Y是非平凡依赖，X必含有码。即每个决定属性集都包含候选码。 也包括了主属性

- [例6.7] 关系模式SJP(S,J,P)中，S是学生，J表示课程，P表示名次。每一个学生选修每门课程的成绩有一定的名次，每门课程中每一名次只有一个学生（即没有并列名次）。
  - 由语义可得到函数依赖： (S,J)→P；(J,P)→S
  - (S,J)与(J,P)都可以作为候选码。
  - 关系模式中没有属性对码传递依赖或部分依赖，所以SJP∈3NF。
  - 除(S,J)与(J,P)以外没有其他决定因素，所以SJP∈BCNF。
- [例6.8] 关系模式STJ(S,T,J)中，S表示学生，T表示教师，J表示课程。每一教师只教一门课。每门课有若干教师，某一学生选定某门课，就对应一个固定的教师。
  - 由语义可得到函数依赖：(S,J)→T；(S,T)→J；T→J
  - 因为没有任何非主属性对码传递依赖或部分依赖，STJ ∈ 3NF。
  - 因为T是决定因素，而T不包含码，所以STJ ∉ BCNF关系。
  - 这里的候选码是(S,J)或者(S,T)



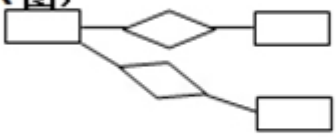
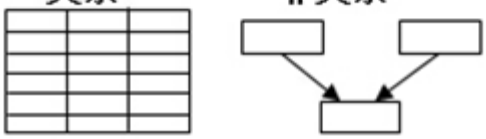
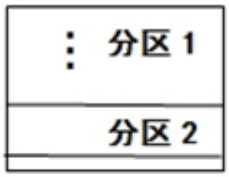
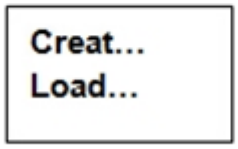
STJ

第七章

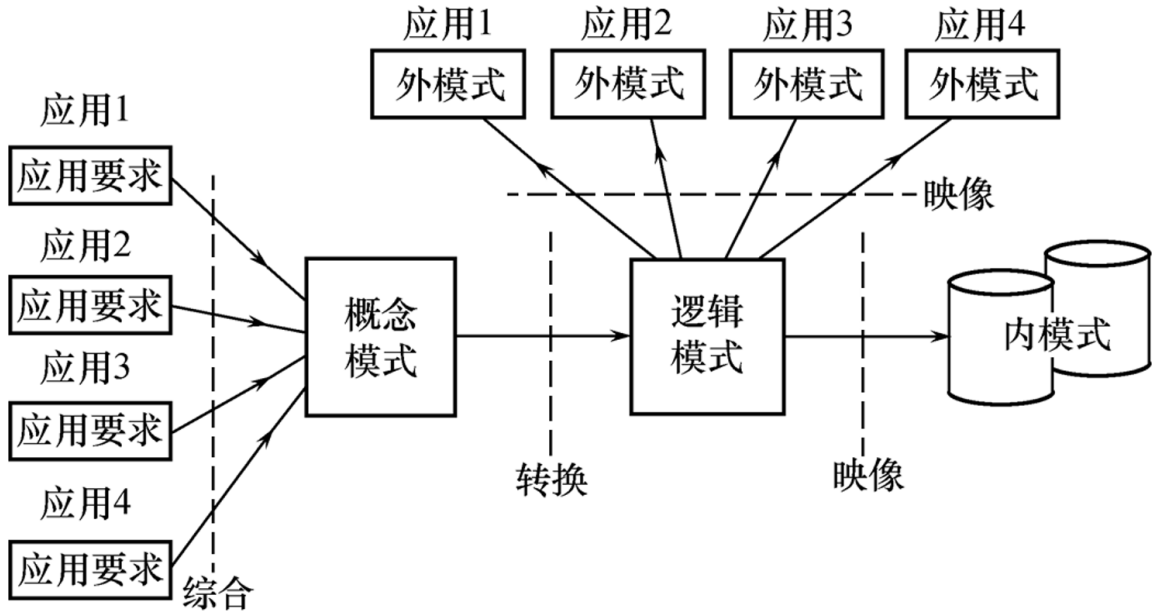
37 数据库设计概述

1. 什么是数据库设计？
    1. 对于一个给定的应用环境，构造（设计）优化的数据库逻辑模式和物理结构，并据此建立数据库及其应用系统，使之能够有效地存储和管理数据，满足各种用户的应用需求，包括**信息管理要求**和**数据操作要求。**
    2. **目标**：为用户和各种应用系统提供一个**信息基础设施和高效率的运行环境 。**
    3. 要求多方面的知识和技术：
        - 计算机的基础知识
        - 软件工程的原理和方法
        - 程序设计的方法和技巧
        - 数据库的基本知识
        - 数据库设计技术
        - 应用领域的知识
    4. 方法：（x）
        - 手工试凑法
        - 规范设计法
            - 新奥尔良（New Orleans）方法
            - 基于E-R模型的数据库设计方法
            - 3NF（第三范式）的设计方法
            - 面向对象的数据库设计方法
            - 统一建模语言（UML）方法
2. 数据库设计的基本步骤有哪些？（与三个模型相对应）

- 需求分析
- **概念结构设计**：形成概念模型
- 逻辑结构设计：转换为数据模型
- 物理结构设计：选择物理结构
- 数据库实施
- 数据库运行和维护

设计阶段	设计描述
需求分析	数字字典、全系统中数据项、数据结构、数据流、数据存储的描述
概念结构设计	概念模型（E-R 图）  数据字典
逻辑结构设计	某种数据模型 关系 非关系 
物理结构设计	存储安排 存取方法选择 存取路径建立 
数据库实施	创建数据库模式 装入数据 数据库试运行 
数据库运行和维护	性能监测、转储/恢复、数据库重组和重构

3. 数据库设计过程中的各级模式
    1. 需求分析阶段（最左）：综合各个用户的应用需求
    2. 概念设计阶段：形成独立于机器特点，独立于各个数据库管理系统产品的**概念模式**（E-R图）
    3. 逻辑设计阶段：
        1. 首先将E-R图转换成具体的数据库产品支持的数据模型，如关系模型，形成数据库**逻辑模式**
        2. 然后根据用户处理的要求、安全性的考虑，在基本表的基础上再建立必要的视图（View），形成数据的**外模式**
    4. 物理设计阶段（最右）：根据数据库管理系统特点和处理的需要，进行物理存储安排，建立索引，形成数据库**内模式**



38 需求分析

1. 数据字典是关于数据库中数据的描述，即**元数据**，不是数据本身。数据字典在**需求分析阶段**建立，在数据库设计过程中不断修改、充实、完善。
    1. 内容：通过对**数据项和数据结构**的定义来描述**数据流、数据存储**的逻辑内容
        - 数据项（数据的最小组成单位）
        - 数据结构（由若干个数据项组成）
        - 数据流：**数据结构**在系统内传输的**路径。**
        - 数据存储：数据结构**停留或保存**的地方，也是数据流的来源和去向之一。
        - 处理过程：一般用判定表或判定树来描述。
2. 第一阶段收集的基础数据（用数据字典来表达）是下一步进行概念设计的**基础。**

39 概念模型和E-R模型

1. 定义：将需求分析得到的用户需求抽象为信息结构（即概念模型）的过程就是概念结构设计。
2. 实体联系部分见第一章04
3. E-R模型：
    - 联系的度：参与联系的实体型的数目
    - 联系可以有属性
    - 如何画E-R模型？
        - 物资管理涉及以下几个实体。
            - 仓库：属性有仓库号、面积、电话号码；
            - 零件：属性有零件号、名称、规格、单价、描述；
            - 供应商：属性有供应商号、姓名、地址、电话号码、账号；
            - 项目：属性有项目号、预算、开工日期；
            - 职工：属性有职工号、姓名、年龄、职称。

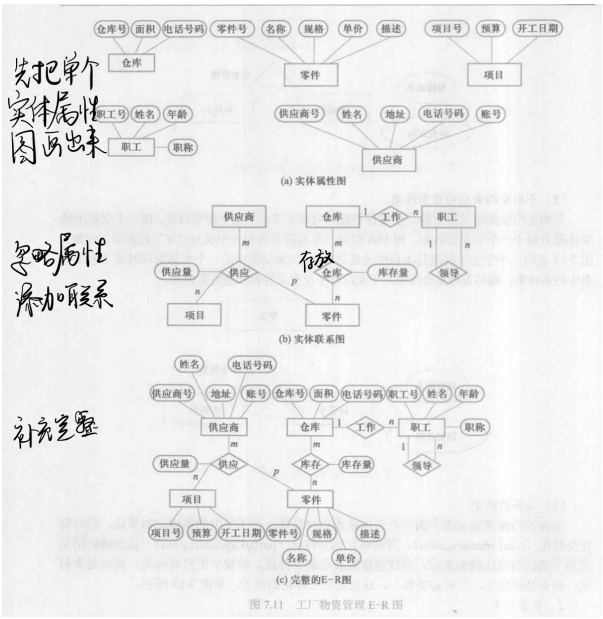
            这些实体之间的联系如下：

            （1）一个仓库可以存放多种零件，一种零件可以存放在多个仓库中，因此仓库和零件具有多对多的联系。用库存量来表示某种零件在某个仓库中的数量。

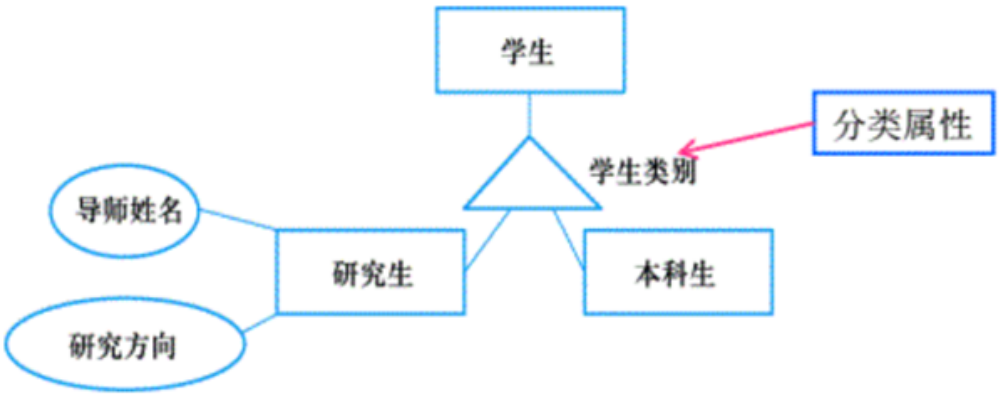
            （2）一个仓库有多个职工当仓库保管员，一个职工只能在一个仓库工作，因此仓库和职工之间是一对多的联系。

            （3）职工之间具有领导与被领导关系，即仓库主任领导若干保管员，因此职工实体型中具有一对多的联系。

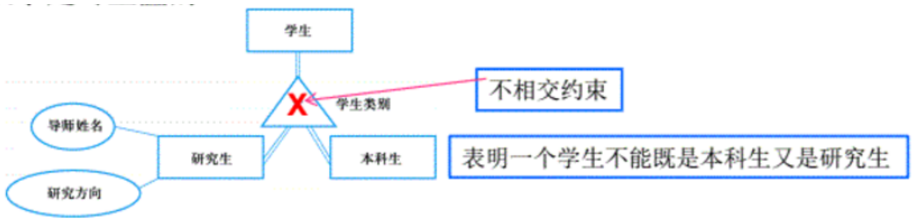
            （4）供应商、项目和零件三者之间具有多对多的联系，即一个供应商可以供给若干项目多种零件，每个项目可以使用不同供应商供应的零件，每种零件可由不同供应商供给。下面给出此工厂的物资管理 E-R 图，如图 7.11 所示。其中，图 7.11(a)为实体属性图，图 7.11(b)为实体联系图，图 7.11(c)为完整的 E-R 图。这里把实体的属性单独画出仅仅是为了更清晰地表示实体及实体之间的联系。

- 

  - ISA联系：父类-子类联系称为ISA联系（下面要掌握）
    - **分类属性**是父实体型的一个属性，其值把父实体型中的实体分派到**子实体型**中



    - 不相交约束：父类中的一个实体**不能同时属于**多个子类中的实体集。

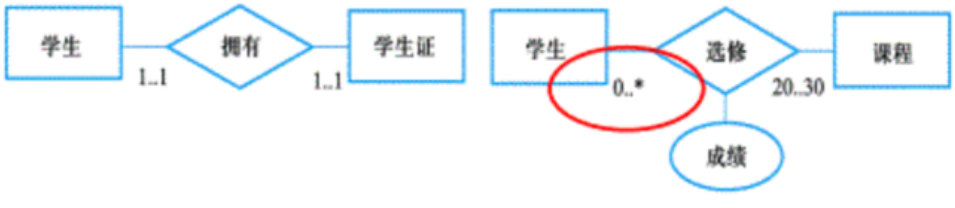


    - 完备性约束：父类中的一个实体是否必须是某一个子类中的实体。若是，则叫做完全特化。用**双线连接**表示。

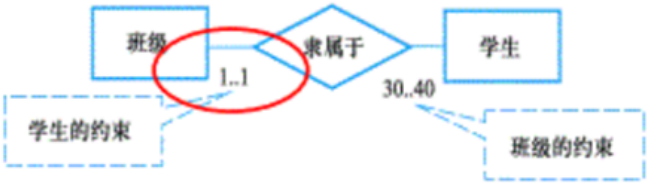


  - 基数约束：最少次数和最多次数。
    - min=1的约束叫做强制参与约束，如每个学生都要属于一个班级
    - min=0的约束叫做非强制参与约束，如0..*

可重叠约束:父类中的一个实体能同时属于多个子类中的实体集。子类符号中没有叉号表示是可重叠的。

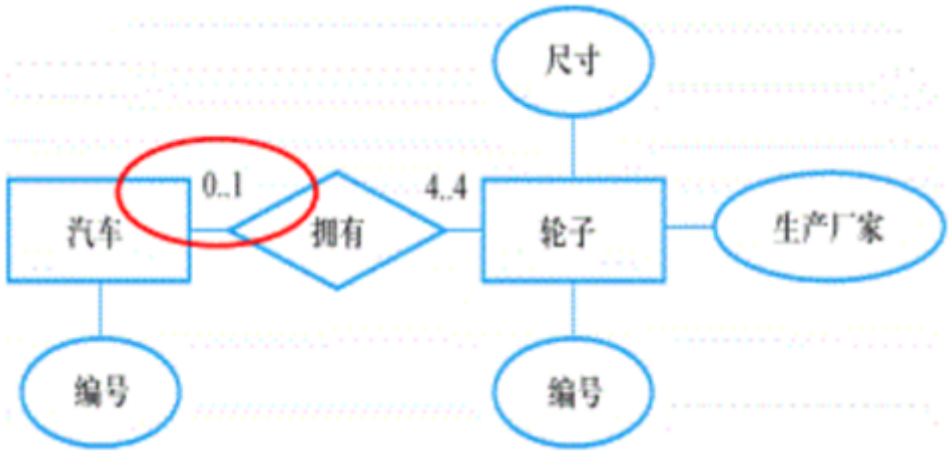


(a) 学生与学生证的联系　　(b) 学生与课程的联系

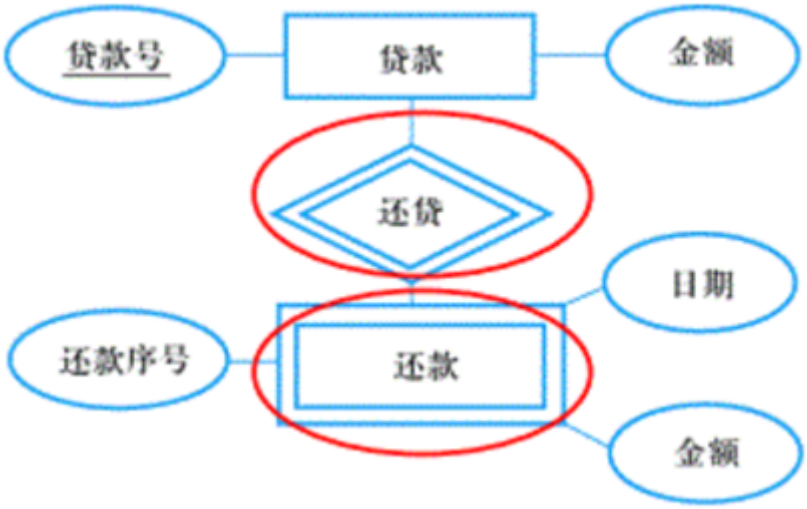


(c) 班级与学生的联系

- Part-of联系：
  - 非独占：整体被破坏，另一部分实体仍可以存在；用0..1即可表示，如汽车和轮子



  - 独占：整体实体如果被破坏，部分实体不能存在；用弱实体类型和识别联系表示
  - **双矩形**表示弱实体型，用**双菱型**表示识别联系，如果贷款没了还款也没了，所以贷款独占还款，还款是弱实体型

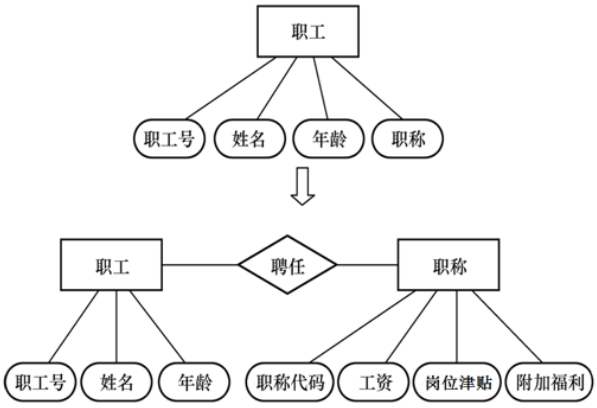


40 概念结构设计

1. 概念结构设计有哪些方法?
    1. 自顶向下
    2. 自底向上

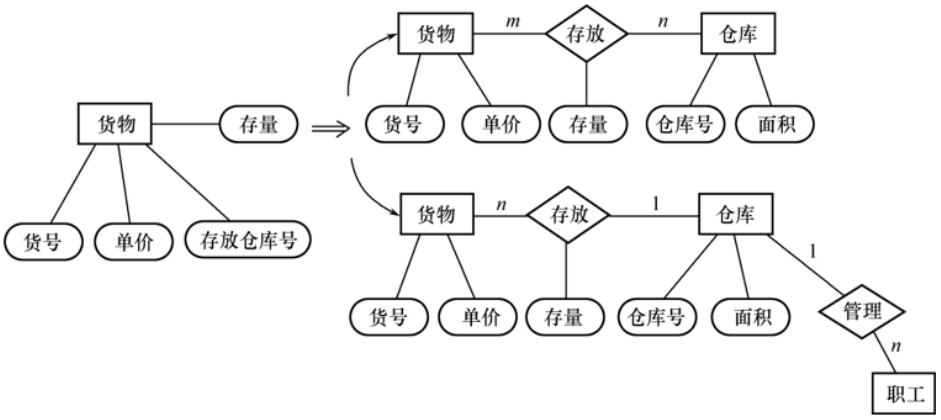
3. 逐步扩张
4. 混合策略
    - 自顶向下地进行需求分析
    - 自底向上地设计概念结构

2. 实体与属性的划分?
    - 属性不可再分，不能与其他实体具有联系
        - [例1] 职工是一个实体，职工号、姓名、年龄是职工的属性。
            - 职称如果没有与工资、福利挂钩，根据准则（1）可以作为职工实体的属性
            - 如果不同的职称有不同的工资、住房标准和不同的附加福利，则职称作为一个实体更恰当



南京大学 软件学院

- [例3] 如果一种货物只存放在一个仓库，那么就可以把存放货物的仓库的仓库号作为描述货物存放地点的属性。如果一种货物可以存放在多个仓库中，或者仓库本身又用面积作为属性，或者仓库与职工发生管理上的联系，那么就应把仓库作为一个实体。



实例：注意订单和订单细节的划分。

（1）每张订单由订单号、若干头信息和订单细节组成。订单细节又有订货的零件号、数量等来描述。按照第二条准则，订单细节就不能作为订单的属性处理而应该上升为实体。一张订单可以订若干产品，所以订单与订单细节两个实体之间是 1:n 的联系。

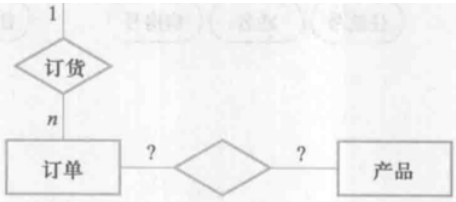


图 7.22　分 E-R 图的框架

（2）原订单和产品的联系实际上是订单细节和产品的联系。每条订货细节对应一个产品描述，订单处理时从中获得当前单价、产品重量等信息。

（3）工厂对大宗订货给予优惠。每种产品都规定了不同订货数量的折扣，应增加一个“折扣规则”实体存放这些信息，而不应把它们放在产品实体中。

最后得到销售管理子系统 E-R 图如图 7.23 所示。

对每个实体定义的属性如下所示。

顾客：{顾客号，顾客名，地址，电话，信贷状况，账目余额}

订单：{订单号，顾客号，订货项数，订货日期，交货日期，工种号，生产地点}

订单细则：{订单号，细则号，零件号，订货数，金额}



1:n为需求分析结果

图 7.23　销售管理子系统的 E-R 图

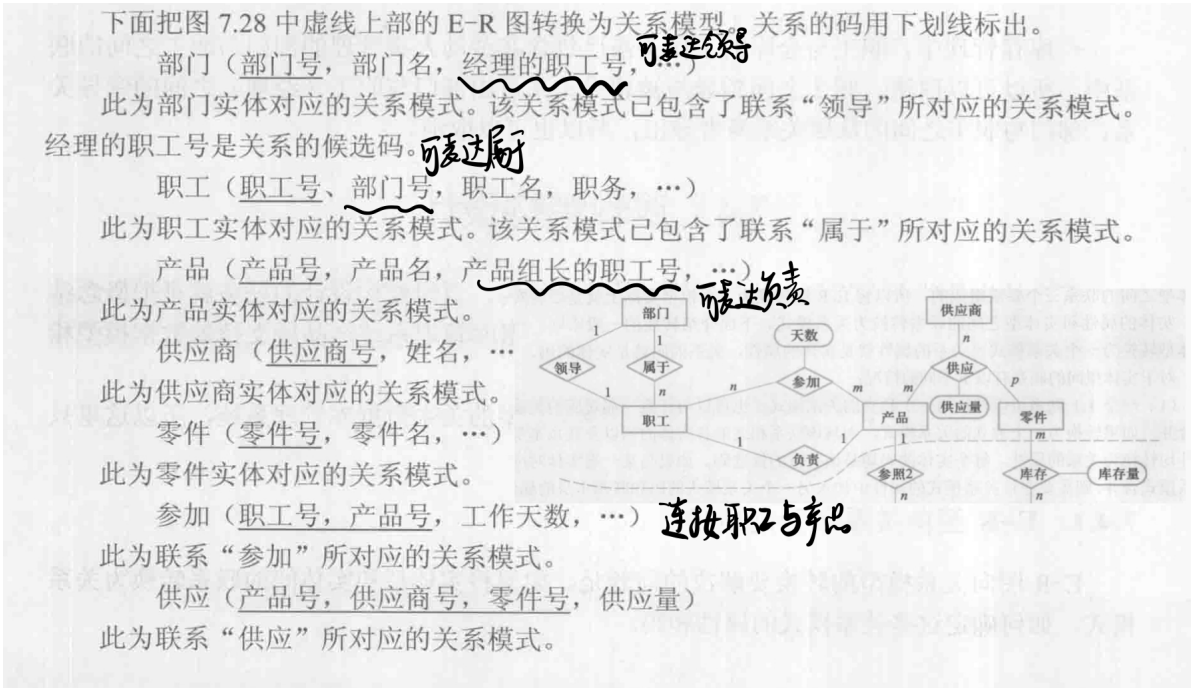
应收账款：{顾客号，订单号，发票号，应收金额，支付日期，支付金额，当前余额，货款限额}

产品：{产品号，产品名，单价，重量}

折扣规则：{产品号，订货量，折扣}

42 逻辑结构设计

1. 任务：将E-R图转换为关系模型
    1. 如图中供应和参加虽然是联系，但有属性，所以作为实体了。



2. 数据模型的优化

- 关系模式的分解
  - 水平分解：把(基本)关系的**元组**分为若干子集合，定义每个子集合为一个子关系。（拿几行出来）
    - 对符合“80/20原则”的，把经常被使用的数据（约20%）水平分解出来，形成一个子关系。
  - 垂直分解：R的**属性分解**为若干子集合，形成若干子关系模式。（拿几列出来）

3. 如何设计外模式（用户子模式）？

   1. 使用更符合用户习惯的别名
   2. 针对不同级别的用户定义不同的视图，以保证系统的安全性。
   3. 简化用户对系统的使用

43 物理结构设计

1. 什么是物理设计?
   - 数据库在物理设备上的**存储结构与存取方法**称为数据库的**物理结构**，它依赖于选定的数据库管理系统。
   - 为一个给定的逻辑数据模型选取一个最适合应用要求的物理结构的过程，就是数据库的物理设计。
2. 怎样进行物理设计?
   - 确定数据库的物理结构，在关系数据库中主要指**存取方法和存储结构**;
   - 对物理结构进行**评价**，评价的重点是时间和空间效率
3. 存取方法如何选择？（原理）
   - B+树索引存取方法
   - Hash索引存取方法
   - 聚簇存取方法

- 为了提高某个属性（或属性组）的查询速度，把这个或这些属性（称为聚簇码）上具有相同值的元组集中存放在连续的物理块中称为聚簇。
- 该属性（或属性组）称为聚簇码（cluster key）

- 聚簇的用途：大大提高按聚簇属性进行查询的效率
  - [例] 假设学生关系按所在系建有索引，现在要查询信息系的所有学生名单。
    - 计算机系的500名学生分布在500个不同的物理块上时，至少要执行500次I/O操作。
    - 如果将同一系的学生元组集中存放，则每读一个物理块可得到多个满足查询条件的元组，从而显著地减少了访问磁盘的次数。
- 聚簇既适用于单个关系，也适用于经常进行连接操作的多个关系
  - 把多个连接的元组按连接属性值聚集存放
  - 从而实现多个关系的“预连接”，提高连接操作的效率。
- 选择聚簇存储方法，即确定需要建立多少个聚簇，每个聚簇中包含哪些关系（一个数据库可以建立多个聚簇，一个关系只能加入一个聚簇）

- 设计候选聚簇
  - 常在一起进行连接操作的关系可以建立组合聚簇
  - 如果一个关系的一组属性经常出现在相等比较条件中，则该单个关系可建立聚簇；
  - 如果一个关系的一个（或一组）属性上的值重复率很高，则此单个关系可建立聚簇。
- 检查候选聚簇中的关系，取消其中不必要的关系
  - 从聚簇中删除经常进行全表扫描的关系
  - 从聚簇中删除更新操作远多于连接操作的关系
  - 从聚簇中删除重复出现的关系
    - 当一个关系同时加入多个聚簇时，必须从这多个聚簇方案（包括不建立聚簇）中选择一个较优的，即在这个聚簇上运行各种事务的总代价最小。

  - 局限性和适合的场景：

- 聚簇只能提高某些特定应用的性能
- 建立与维护聚簇的开销相当大
  - 对已有关系建立聚簇，将导致关系中元组的物理存储位置移动，并使此关系上原有的所有索引无效，必须重建。
  - 当一个元组的聚簇码改变时，该元组的存储位置也要做相应改变。
- 当通过聚簇码进行访问或连接是该关系的主要应用，与聚簇码无关的其他访问很少或者是次要的时，可以使用聚簇
  - 尤其当SQL语句中包含有与聚簇码有关的ORDER BY, GROUP BY, UNION, DISTINCT等子句或短语时，使用聚簇特别有利，可以省去或减化对结果集的排序操作

4. 存储结构如何确定？受硬件环境和应用需求的影响。

44 数据库的实施和维护x

组织数据入库是数据库实施阶段最主要的工作。

试运行：功能测试和性能测试，测量性能指标。

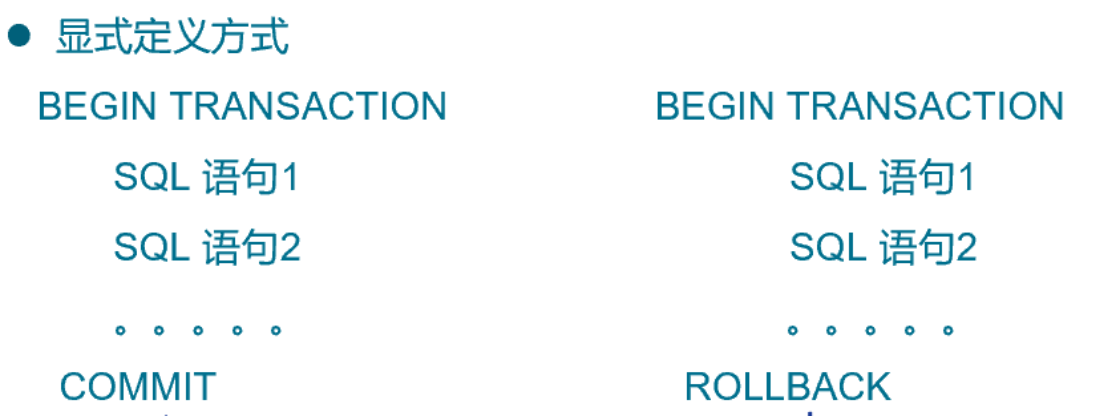
维护：

- 重组织（全部或部分）：不会改变原设计的**数据逻辑结构和物理结构**，一般DBMS会提供实用程序
- 重构造：根据新环境调整数据库的**模式和内模式**

第十章

48 事务

1. 什么是事务?
    1. 事务(Transaction)是用户定义的一个数据库操作序列，这些操作**要么全做，要么全不做**，是一个不可分割的工作单位。
    2. 事务和程序是两个概念
        - 在关系数据库中，一个事务可以是**一条SQL语句，一组SQL语句或整个程序**
        - 一个程序通常包含多个事务
    3. 事务是**恢复和并发控制**的基本单位
2. 事务的定义：



    - COMMIT提交事务的**所有操作**（读+更新），事务中所有对数据库的更新写回到**磁盘上的物理数据库**中
    - ROLLBACK系统将事务中对数据库的所有已完成的操作**全部撤销**，事务滚回到开始时的状态
3. ACID特性(*)：
    1. 原子性：事务是基本单位，要么都做要么都不做
    2. 一致性：
        1. 若一致，数据库中**只包含**成功事务提交的结果；
        2. 若不一致，数据库系统运行中发生**故障**，有些事务尚未完成就被迫中断；这些未完成事务对数据库所做的修改有一部分已写入**物理数据库**，这时数据库就处于一种不正确的状态
    3. 隔离性：一个事务的执行不被其他并发事务**干扰**
    4. 持续性：一个事务一旦提交，它对数据库中数据的改变就应该是**永久**性的
4. 破坏ACID的因素?
    1. 多个事务并行运行时，不同事务的操作**交叉执行**
    2. 事务在运行过程中被**强行停止**

49 故障和数据库恢复

1. 故障的种类
    1. 事务内部的故障
    2. 系统故障

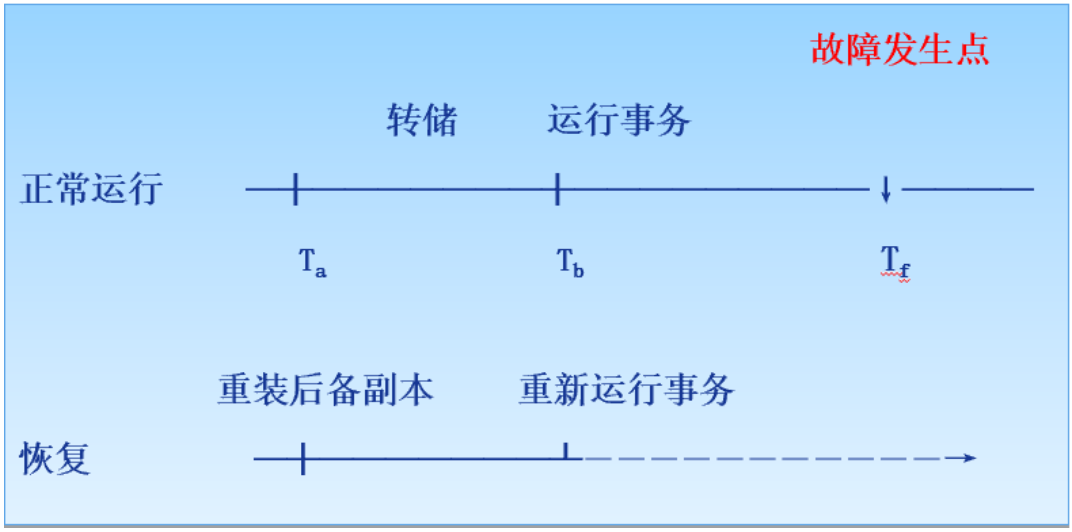
    3. 介质故障
    4. 计算机病毒
2. 事务故障（仅指以下的非预期故障）
    1. 运算**溢出**
    2. 并发事务发生死锁而被选中**撤销该事务**
    3. 违反了某些**完整性限制**而被终止等
    4. 通过**事务撤销**（UNDO）解决：**强行回滚**（ROLLBACK）该事务，撤销该事务已经作出的任何对数据库的修改，使得该事务像根本没有启动一样
3. 系统故障：软故障，不破坏数据库
    1. 一些尚未完成的事务的结果可能**已送入物理数据库**，造成数据库可能处于不正确状态。
        1. 恢复策略：系统重新启动时，恢复程序让**所有非正常终止的事务回滚**，强行撤消（UNDO）所有未完成事务
    2. 发生系统故障时，有些已完成的事务可能有一部分甚至全部留在**缓冲区**，尚未写回到磁盘上的物理数据库中，系统故障使得这些事务对数据库的修改部分或全部丢失
        1. 恢复策略：系统重新启动时，恢复程序需要**重做**（REDO）所有已提交的事务
4. 介质故障：硬故障，破坏数据库或部分数据库，影响正在存取这部分数据的**所有事务**
5. 计算机病毒：破坏数据库

故障类型	**恢复策略**
事务故障	强行回滚该事务
系统故障	1.强行撤销所有**未完成**事务 2.重做所有**已提交**的事务

6. 恢复操作：
    1. 基本原理：冗余。利用存储在系统别处的**冗余数据**来重建数据库中已被破坏或不正确的那部分数据
    2. 关键问题：
        1. 如何建立冗余数据（数据转储，登记日志文件）
        2. 如何利用冗余数据进行恢复

50 数据转储和日志文件

1. 数据转储
    1. 数据库管理员定期地将整个数据库**复制**到磁带、磁盘或其他存储介质上保存起来的过程
    2. 只能将数据库恢复到转储时的状态，要想恢复到故障发生时的状态，必须重新运行**自转储以后的所有更新事务**



运行自Tb ~ Tf时刻的所有更新事务

3. 转储类型
    - 静态转储：在系统中**无运行事务**时进行的转储操作，转储期间**不允许**对数据库的任何存取、修改活动，得到的一定是一个数据一致性的副本
    - 动态转储：转储操作与用户事务**并发**进行，需要把**动态转储期间**各事务对数据库的修改活动登记下来，建立**日志文件**
    - 海量转储：每次转储**全部**数据库
    - 增量转储：只转储上次转储后**更新过**的数据

2. 日志文件：
    1. 用来记录事务对数据库的更新操作的文件
    2. 格式：以**记录**为单位；

- 以记录为单位的日志文件内容
    - 日志文件中的一个日志记录 (log record)包含
        - 各个事务的开始标记(BEGIN TRANSACTION)
        - 各个事务的结束标记(COMMIT或ROLLBACK)
        - 各个事务的所有更新操作

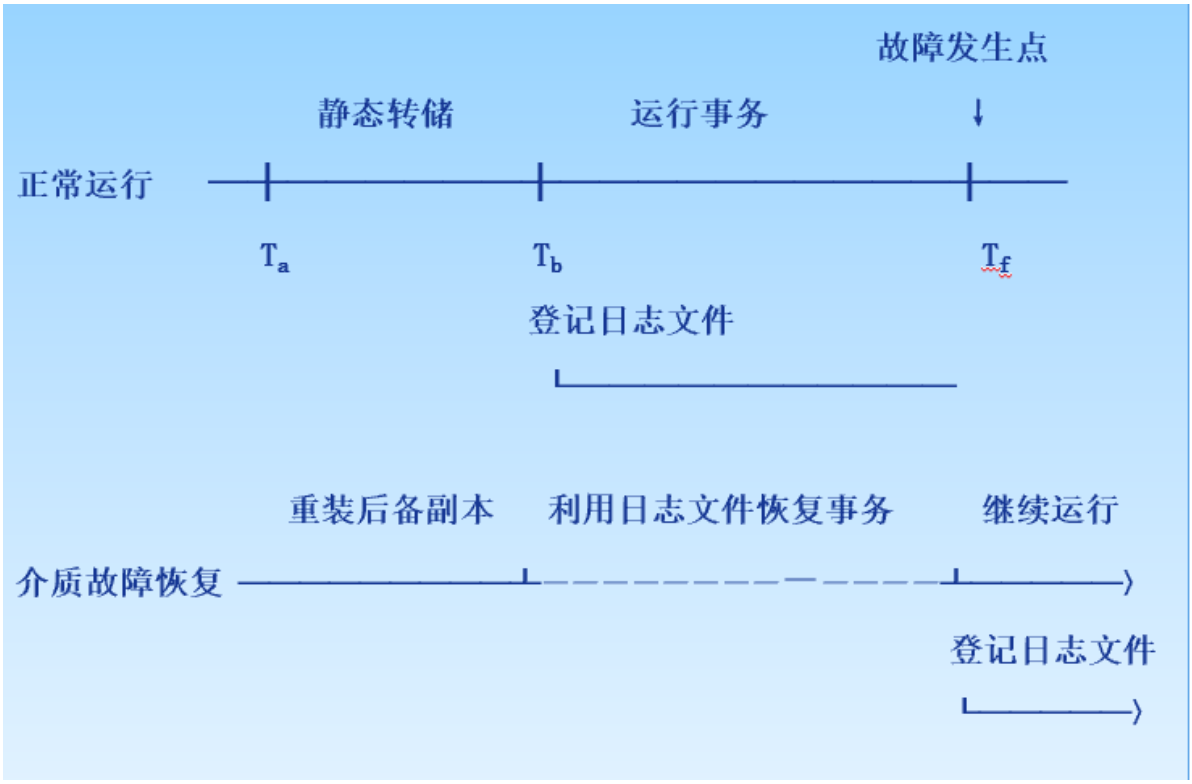
- 以记录为单位的日志文件，每条日志记录的内容
    - 事务标识（标明是哪个事务）
    - 操作类型（插入、删除或修改）
    - 操作对象（记录内部标识）
    - 更新前数据的旧值（对插入操作而言，此项为空值）
    - 更新后数据的新值（对删除操作而言，此项为空值）

以**数据块**为单位；

- 以数据块为单位的日志文件，每条日志记录的内容
    - 事务标识
    - 被更新的数据块

    3. 用途：进行事务故障恢复，进行系统故障恢复，协助后备副本（转储）进行介质故障恢复

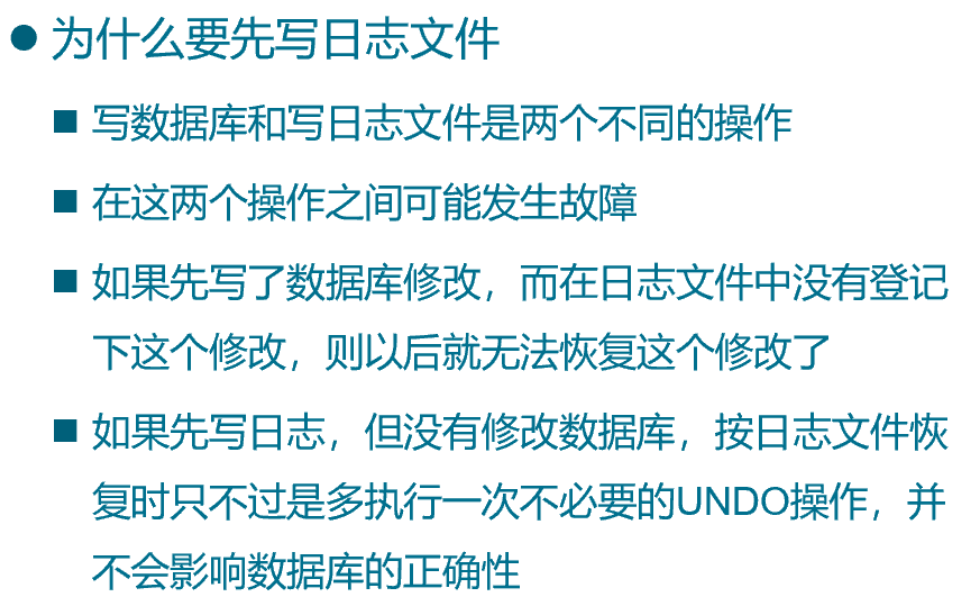
必须使用日志文件	可以建立日志文件
事务故障	静态转储（利用日志文件重做已完成，撤销未完成）
系统故障	
动态转储	



- 可以选择重做或者登记日志文件

    4. 登记日志文件的原则：

        1. 登记的次序严格按并发事务执行的**时间次序**
        2. 必须先写日志文件，后写数据库



- 中途有故障，说明该事务未完成，会利用日志文件进行撤销，那多余的UNDO不影响的

51 恢复策略

1. 事务故障的恢复：恢复到**最开始**，系统自动完成
    1. 反向扫描文件日志
    2. 对该事务的**更新操作**执行**逆操作**。即将日志记录中“更新前的值” 写入数据库。
    3. 继续反向扫描日志文件，查找该事务的其他**更新操作**，并做同样处理。
    4. 如此处理下去，直至读到此事务的**开始标记**，事务故障恢复就完成了。
2. 系统故障的恢复：恢复到故障时

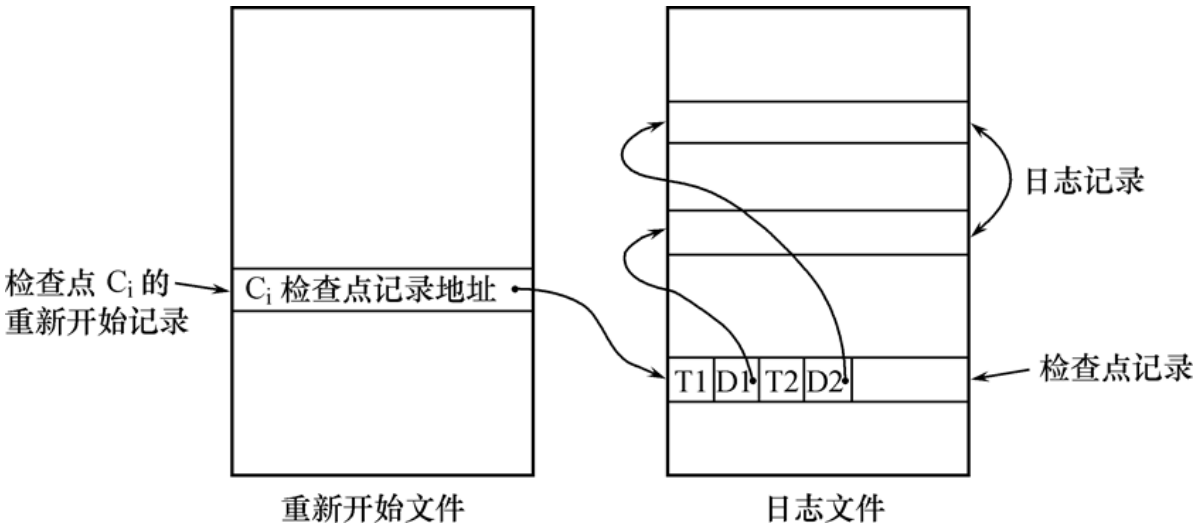
1. **正向扫描日志文件**（即从头扫描日志文件），建立重做(REDO)队列和撤销(UNDO)队列
2. 对撤销(UNDO)队列事务进行**撤销**(UNDO)处理：反向扫描日志文件，执行逆操作
3. 对重做(REDO)队列事务进行**重做**(REDO)处理：正向扫描日志文件，重新执行登记的操作

3. 介质故障的恢复：

1. **装入最新的后备数据库副本**(离故障发生时刻最近的转储副本)，使数据库恢复到最近一次转储时的一致性状态
2. 装入有关的日志文件副本(转储结束时刻的日志文件副本)，**重做已完成的事务。**

52 具有检查点的恢复技术

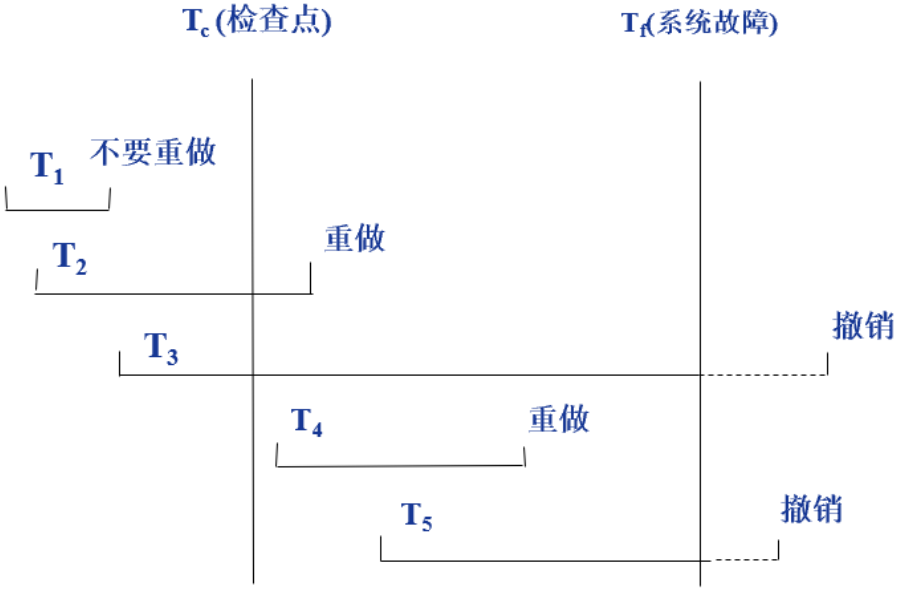
- 恢复的两个问题
  - 搜索整个日志将耗费大量的时间
  - 重做处理：重新执行，浪费了大量时间
- 具有检查点（checkpoint）的恢复技术
  - 在日志文件中增加检查点记录（checkpoint）
  - 增加重新开始文件
  - 恢复子系统在登录日志文件期间动态地维护日志



- 动态维护日志文件的方法
  - 周期性地执行如下操作：建立检查点，保存数据库状态。
  - 具体步骤是：
    - 将当前日志缓冲区中的所有日志记录写入磁盘的日志文件上
    - 在日志文件中写入一个检查点记录
    - 将当前数据缓冲区的所有数据记录写入磁盘的数据库中
    - 把检查点记录在日志文件中的地址写入一个重新开始文件

恢复策略：就是看从检查点到故障中间有没有提交，有提交就重做（防止已完成未写入），没提交就撤销（防止未完成在缓存）

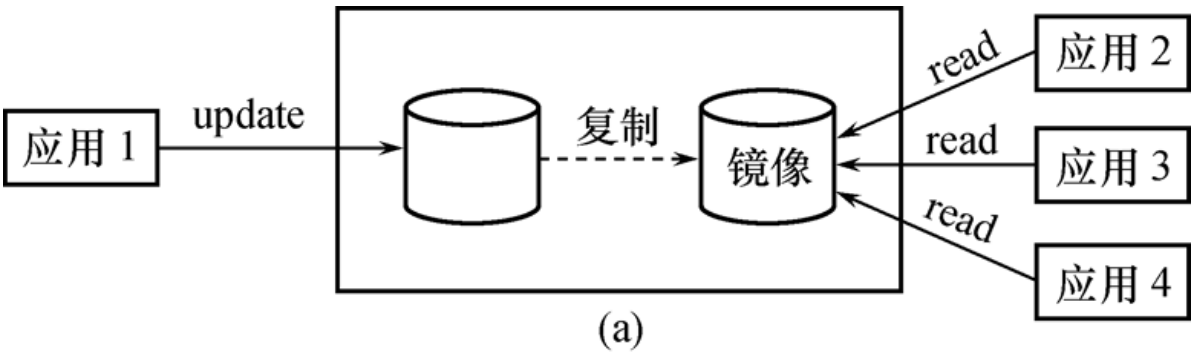
系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略



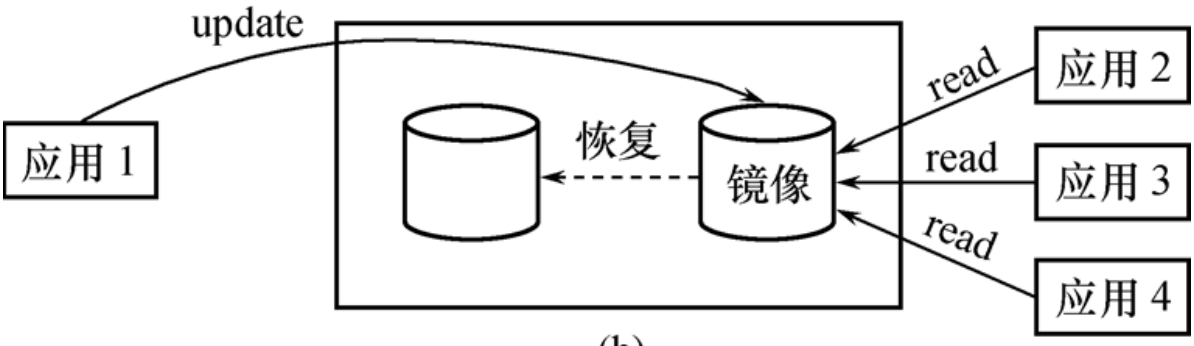
1. 检查点的作用？使用检查点方法可以**改善恢复效率**
   - 节约了搜索日志和重做的时间

> - 当事务T在一个检查点之前提交，T对数据库所做的修改已写入数据库
> - 写入时间是在这个检查点建立之前或在这个检查点建立之时
> - 在进行恢复处理时，没有必要对事务T执行重做操作

53 数据库镜像x



(a)



(b)

数据库镜像：数据库管理系统**自动**把整个数据库或其中的关键数据复制到另一个磁盘上

- 出现介质故障时，可由镜像磁盘**继续提供使用**，同时数据库管理系统**自动**利用镜像磁盘数据进行数据库的**恢复**，不需要关闭系统和重装数据库副本。
- 没有出现故障时，可用于**并发操作。**一个用户对数据加排他锁修改数据，其他用户可以**读镜像数据库上的数据**，而不必等待该用户释放锁

频繁地复制数据自然会降低系统运行效率，所以在实际应用中用户往往只选择对**关键数据和日志文件**镜像，不是对整个数据库进行镜像。

第十一章

54 并发控制

1. 事务串行执行：每个时刻**只有一个**事务运行，其他事务必须等到这个事务结束以后方能运行
2. 交叉并发方式：因为是单处理机，并没有真正地并行运行，但能够减少处理机的空闲时间，提高系统的效率
3. 同时并发方式（多处理机系统）：每个处理机可以运行一个事务，**多个处理机可以同时运行多个事务**，实现多个事务真正的并行运行。最理想的并发方式，但受制于硬件环境。

并发控制机制的任务?

1. 对并发操作进行**正确调度**
2. 保证事务的**隔离性和一致性**

并发操作带来的数据不一致性：

1. 丢失修改（Lost Update）
    - 如T2的提交结果**破坏了**（覆盖了）T1提交的结果，导致T1的修改被丢失。
2. 不可重复读（Non-repeatable Read）
    - 事务T1读取数据后，事务T2执行更新操作，使T1无法**再现前一次**读取结果。**校验**时就会出错。
        - T2对其做了修改

        - T2删除了其中部分记录
        - T2插入了一些记录
3. 读“脏”数据（Dirty Read）
    - 事务T1修改某一数据，并将其写回磁盘
    - 事务T2读取同一数据后， T1由于某种原因被**撤销**ROLLBACK
    - 这时T1已修改过的数据恢复原值， T2读到的数据就与数据库中的数据不一致
    - T2读到的数据就为“脏”数据，即**不正确**的数据

造成上述三类数据不一致性的主要原因是并发操作破坏了事务的隔离性。

并发控制就是要用**正确的方式调度并发操作**，使一个用户事务的执行不受其他事务的**干扰**，从而**避免**造成数据的不一致性

并发控制的主要技术?

1. 封锁(Locking)
2. 时间戳(Timestamp)
3. 乐观控制法
4. 多版本并发控制(MVCC)

55 封锁

封锁就是事务T在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其**加锁**（对数据对象加锁）。

加锁后事务T就对该数据对象有了一定的控制，在事务T释放它的锁之前，其它的事务**不能更新**此数据对象。

- 排它锁：若事务T对数据对象A加上**X锁**，则只允许T**读取和修改**A，其它任何事务都不能再对A加任何类型的锁，直到T释放A上的锁
    - 保证其他事务在T释放A上的锁之前**不能再读取和修改A** （独占标记）
- 共享锁：若事务T对数据对象A加上S锁，则事务T可以**读A但不能修改A**，其它事务只能再对A加S锁，而不能加X锁，直到T释放A上的S锁
    - 保证其他事务**可以读A**，但在T释放A上的S锁之前不能对A做任何**修改** （大家都能读但不能改）

T_2 \ T_1	X	S	–
X	N	N	Y
S	N	Y	Y
–	Y	Y	Y

封锁协议

- 何时申请X锁或S锁
- 持锁时间
- 何时释放

1. 一级封锁协议

1. 事务T在修改数据R之前必须先对其加X锁，直到事务结束才释放。包括正常结束（COMMIT）和非正常结束（ROLLBACK）

2. 二级封锁协议
    1. 一级封锁协议基础上增加事务T在读取数据R之前必须先对其加S锁，**读完后**即可释放S锁。
3. 三级封锁协议
    1. 一级封锁协议基础上加上事务T在读取数据R之前必须先对其加S锁，直到**事务结束**才释放。

	写	读	解决	未解决
一级封锁协议	加X锁	不加锁	丢失修改	不可重复读、读“脏”数据
二级封锁协议	加X锁	加S锁，**读完后**释放	丢失修改，读“脏”数据	不可重复读
三级封锁协议	加X锁	加S锁，**事务结束后**释放	丢失修改、读脏数据、不可重复读	无

封锁协议小结

- 三级协议的主要区别
  - 什么操作需要申请封锁以及何时释放锁（即持锁时间）
- 不同的封锁协议使事务达到的一致性级别不同
  - 封锁协议级别越高，一致性程度越高

	X锁		S锁		一致性保证		
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不读“脏”数据	可重复读
一级封锁协议		√			√		
二级封锁协议		√	√		√	√	
三级封锁协议		√		√	√	√	√

活锁与死锁

1. 避免活锁：先来先服务。当多个事务请求封锁同一数据对象时，按**请求封锁的先后次序**对这些事务排队，该数据对象上的锁一旦释放，首先批准申请队列中**第一个事务**获得锁

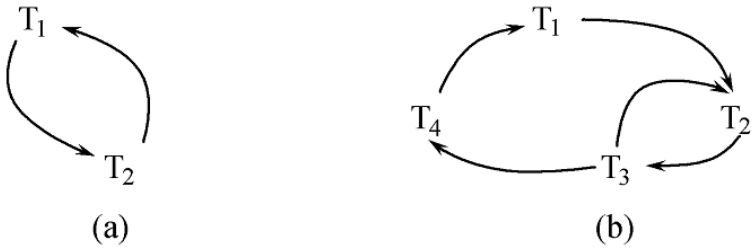
2. 死锁：为什么不释放自己的锁？因为**事务结束**时才能释放。

T_1	T_2
Lock R_1	•
	•
	•
•	Lock R_2
•	•
•	•
Lock R_2	•
等待	
等待	
等待	Lock R_1
等待	等待
等待	等待
	•
	•
	•

   - 预防：
     - 一次封锁法：要求每个事务必须**一次将所有要使用的数据**全部加锁，否则就不能继续执行
     - 顺序封锁法：预先对数据对象**规定一个封锁顺序**，所有事务都按这个顺序实行封锁。
   - 诊断并解除死锁：更普遍采用
     - 超时法：如果一个事务的等待时间超过了**规定的时限**，就认为发生了死锁

可能误判死锁（时限过短），也可能识别不出死锁（时限过长）

     - 等待图法：**并发控制**子系统周期性地（比如每隔数秒）生成事务等待图，检测事务。如果发现图中**存在回路**，则表示系统中出现了死锁。
       - 结点为正在运行的事务，边为事务等待的情况



       -
         - 图(a)中，事务T_1等待T_2，T_2等待T_1，产生了死锁
         - 图(b)中，事务T_1等待T_2，T_2等待T_3，T_3等待T_4，T_4又等待T_1，产生了死锁
         - 图(b)中，事务T_3可能还等待T_2，在大回路中又有小的回路
     - 选择一个**处理死锁代价最小**的事务，将其**撤消**；**释放此事务持有的所有的锁**，使其它事务能继续运行下去

56 事务调度

1. 可串行化(Serializable)调度
   - 多个事务的并发执行是正确的，当且仅当其结果与按某一次序**串行地执行**这些事务时的结果相同（即和**不并发**得到的结果一样，显然就得到了正确的结果）
2. 可串行性(Serializability)
   - 是并发事务正确调度的准则
   - 一个给定的并发调度，当且仅当它是**可串行化**的，才认为是正确调度
3. 冲突可串行化（比可串行化更严格）
   1. 冲突操作：仅指T1、T2对同一数据的**读写**操作和**写写**操作

2. 不能交换（Swap）的动作：同一事务的两个操作和不同事务的冲突操作
3. 定义：一个调度Sc在保证冲突操作的次序不变的情况下，通过交换**两个事务不冲突操作的次序**得到另一个调度Sc'，如果Sc'是**串行**的，称调度Sc是冲突可串行化的调度

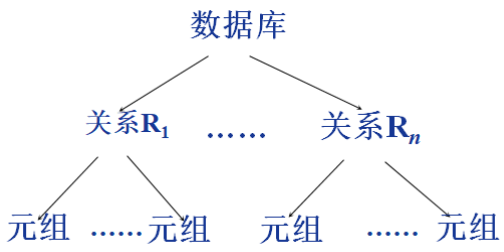
此处的串行是本身就是串行，而非具有可串行性

两段锁协议

1. 实现并发调度的可串行性，从而保证调度的正确性
2. 指所有事务必须分**两个阶段**对数据项加锁和解锁
   - 在对任何数据进行读、写操作之前，事务首先要获得对该数据的**封锁**：扩展阶段
   - 在**释放**一个封锁之后，事务不再申请和获得任何其他封锁：收缩阶段
3. "两段"锁的含义，事务分为两个阶段
   - 第一阶段是获得封锁，也称为扩展阶段
     - 事务可以**申请**获得任何数据项上的任何类型的锁，但是**不能释放**任何锁
   - 第二阶段是释放封锁，也称为收缩阶段
     - 事务可以**释放**任何数据项上的任何类型的锁，但是**不能再申请**任何锁
4. 若并发事务都遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的；若并发事务的一个调度是可串行化的，**不一定**所有事务都符合两段锁协议。
5. 两段锁协议与防止死锁的一次封锁法：
   - 一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此一次封锁法**遵守两段锁协议**（释放后没得再加的东西了）
   - 两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此遵守两段锁协议的事务**可能发生死锁**

57 封锁粒度

1. 封锁对象的大小称为封锁粒度(Granularity)
2. 封锁的对象：
   1. 逻辑单元：属性值、属性值的集合、元组、关系、索引项、整个索引、整个数据库等
   2. 物理单元：页（数据页或索引页）、物理记录等
3. 封锁粒度与系统的并发度和并发控制的开销密切相关。
   1. 封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；
   2. 封锁的粒度越小，**并发度较高**，但系统**开销也就越大**
   3. 所以选择时同时考虑**封锁开销**和**并发度**
      1. 需要处理**多个关系**的大量元组的用户事务：以数据库为封锁单位
      2. 需要处理大量元组的用户事务：以**关系**为封锁单元
      3. 只处理少量元组的用户事务：以元组为封锁单位
4. 多粒度封锁：在一个系统中同时支持多种封锁粒度供不同的事务选择
   1. 首先要定义多粒度树。如三级粒度树：



   2. 多粒度封锁协议：对一个结点加锁意味着这个结点的所有后裔结点也被加以同样类型的锁。因此在多粒度封锁中一个数据对象可能以两种方式封锁：
      - 显式封锁：直接加到**数据对象**上的封锁

- 隐式封锁：是该数据对象没有独立加锁，是由于其**上级结点**加锁而使该数据对象加上了锁

3. 所以检查封锁冲突时显式和隐式都要检查。即该数据对象，所有上级节点，所有下级结点都要检查。

    1. 例如，事务T要对关系R1加X锁：

        - 系统必须搜索其**上级结点**数据库、关系R1
        - 还要搜索R1的**下级结点**，即R1中的每一个元组
        - 如果其中某一个数据对象已经加了不相容锁，则T必须等待

意向锁：提高对某个数据对象加锁时系统的检查效率

1. 如果对一个结点加意向锁，则说明该结点的下层结点正在被加锁
2. 对任一结点加基本锁，必须先对它的上层结点加意向锁

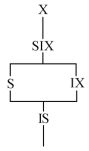
常见意向锁	内容
意向共享锁IS锁	表示它的后裔结点拟（意向）加S锁
意向排它锁IX锁	表示它的后裔结点拟（意向）加X锁
共享意向排它锁SIX锁	如果对一个数据对象加SIX锁，表示对它加S锁，再加IX锁，即SIX = S + IX

T_1 \ T_2	S	X	IS	IX	SIX	–
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
–	Y	Y	Y	Y	Y	Y

Y=Yes，表示相容的请求　　N=No，表示不相容的请求

(a) 数据锁的相容矩阵

锁的强度：



(b) 锁的强度的偏序关系

申请时自上而下检查，释放时自下而上

NoSQL

63 NoSQL的由来

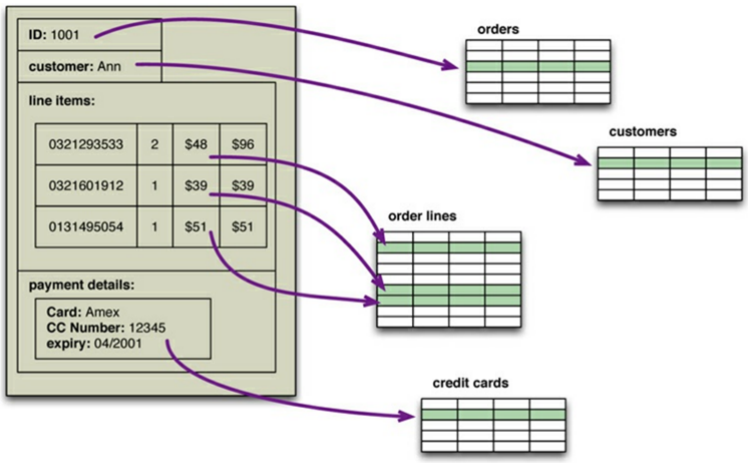


Figure 1.1. An order, which looks like a single aggregate structure in the UI, is split into many rows from many tables in a relational database

1. **阻抗失谐**： 关系模型和内存中的数据结构之间存在差异
    1. 如果在内存中使用了较为丰富的数据结构，那么要把它保存到磁盘之前，必须先将其转换成关系形式（关系+元组）。
    2. 于是就发生了“阻抗失谐”：需要在**两种不同的表示形式**之间转译。但关系型数据库经受住了考验，而面向对象数据库销声匿迹。
    3. 解决方法：“对象-关系映射框架”通过**映射模式**表达转换。但仍有**查询性能问题和集成问题**。
2. 关系型数据库战胜面向对象数据库的原因？SQL充当了应用程序之间的一种**集成**机制。数据库在这种情况下成了“集成数据库”
3. 但也可以将数据库视为“应用程序数据库”， 其内容只能由**一个**应用程序的代码库直接访问
    - 区别：
        - 前者由不同团队所开发的**多个**应用程序，将其数据存储在一个**公用**的数据库中。为了保持数据库的完整性，我们需要将更新数据的责任交由**数据库自身**负责。
        - 后者中一个应用程序对应一个数据库，由于只有开发应用程序的团队才需要知道其结构，模式的维护与更新就更容易了。由于应用程序开发团队同时管理数据库和应用程序代码，因此可以把维护数据库完整性的工作放在**应用程序代码**中。
    - 在使用应用程序数据库后，由于内部数据库与外部通信服务之间已经解耦，所以外界并不关心数据如何存储，这样就可以选用**非关系型数据库**了。关系型数据库的许多特性，诸如安全性等，可以交给使用该数据库的外围应用程序来做。
4. 集群问题：纵向扩展及横向扩展来应对数据和流量的增加。而采用**多个小型计算机组成的集群**是一种解决方案。
    - 但关系型数据库不是给集群用的。
    - 1.用共享磁盘子系统”，但磁盘子系统就变成了整个集群的软肋。
    - 2.将数据划分为几个集合。在**负载分散**的同时，应用程序必须控制所有**分片**，需要知道数据库中的每份小数据的存储情况。而且跨分片的查询、参照完整性、 事务、一致性控制等操作也都无法以跨分片的方式执行。
    - 总之怎么弄都不太协调。

- 而面向聚合的数据库非常适用于横向扩展。

5. NoSQL：开源分布式的非关系型数据库。特征有
    1. 不使用SQL
    2. 在集群中运行良好：关系型数据库使用ACID事务来保持整个数据库的**一致性**，而这种方式本身与集群环境相冲突。NoSQL数据库为处理**并发及分布问题**提供了众多选项。
    3. 开源
    4. 适用于21世纪的互联网公司
    5. 无模式：不用事先修改结构定义，即可自由添加字段了

64 聚合

1. 把一组相互关联的对象视为**一个整体单元**来操作，而这个单元就叫聚合。比如键值、文档、列族、图等。

- 通过**原子操作**更新聚合的值（含一致性管理）
- 以**聚合**为单位与数据存储通信
- 在集群中操作数据库时，用聚合为单位来**复制和分片**
- 由于程序员经常通过聚合结构来操作数据，故而采用聚合也能让其工作更为轻松。

2. **聚合无知**：关系型数据库的数据模型中，没有“聚合”这一概念。“图数据库"也是聚合无知的。
    1. 采用“聚合无知”模型，那么很容易就能以不同方式来查看数据。在操作数据时，如果没有一种占主导地位的结构，那么选用此模型效果会更好，如关系型数据库和图数据库。
    2. 聚合的边界难以划分。
3. 选用的决定性因素：面向聚合模型非常适合在**集群**上运行，可以把节点数降到最小。
4. 面向聚合的数据库不支持多个聚合的ACID事务，只能保证单一聚合内部内容的原子性（在单一操作中更新跨越多张表的数个行，要么完全成功，要么彻底失败），即**每次只能在一个聚合结构上执行原子操作**。如果想以原子方式操作多个聚合，那么就必须自己组织应用程序的代码。

65 主要的NoSQL数据模型

1. **键值数据模型**
    1. 键值数据库的聚合**不透明**，只包含一些没有太多意义的大块信息
        - 聚合中可以存储**任意数据**。数据库可能会限制聚合的总大小，但除此之外，其他方面都很随意
        - 在键值数据库中，要访问聚合内容，只能通过**键**来查找
2. **文档数据模型**
    1. 在文档数据库的聚合中，可以看到其**结构**。
        - 限制其中存放的内容，它定义了其允许的结构与数据类型
        - 能够更加灵活地访问数据。通过用聚合中的**字段查询**，可以只获取一部分聚合，而不用获取全部内容
        - 可以按照聚合内容创建索引
3. **列族存储**
    1. 大部分数据库都以行为单元存储数据。然而，有些情况下写入操作执行得很少，但是经常需要一次读取若干行中的很多列。此时，列存储数据库将**所有行的某一组列**作为基本数据存储单元。
    2. 两级聚合结构：get('1234','name')="martin"

3. 设计前提：某个列族中的数据经常需要**一起访问**。

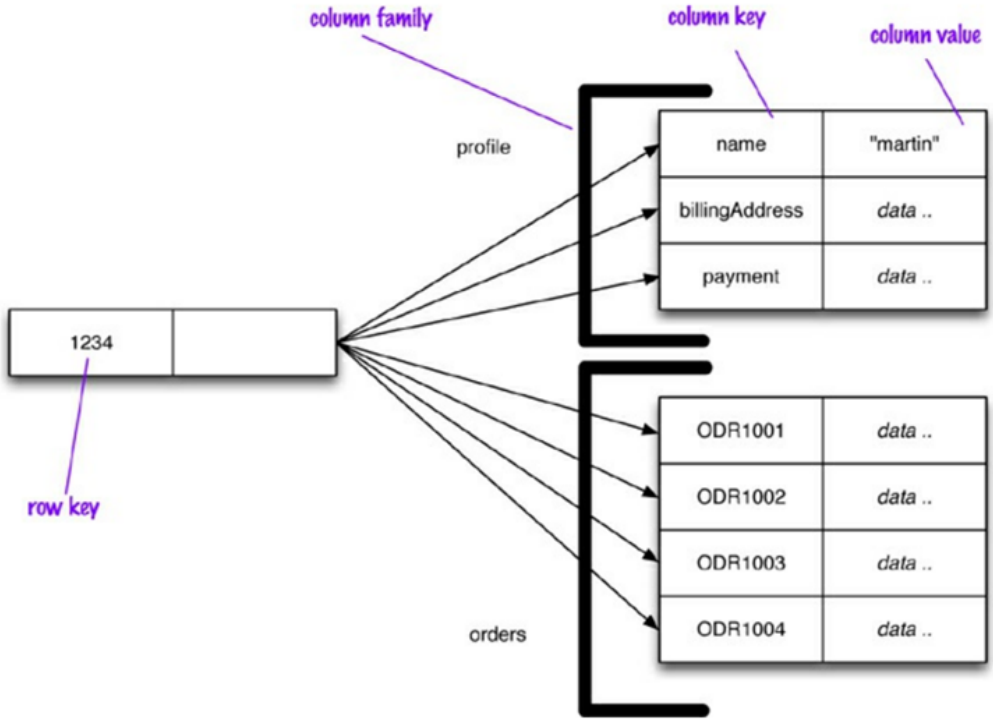


Figure 2.5. Representing customer information in a column-family structure

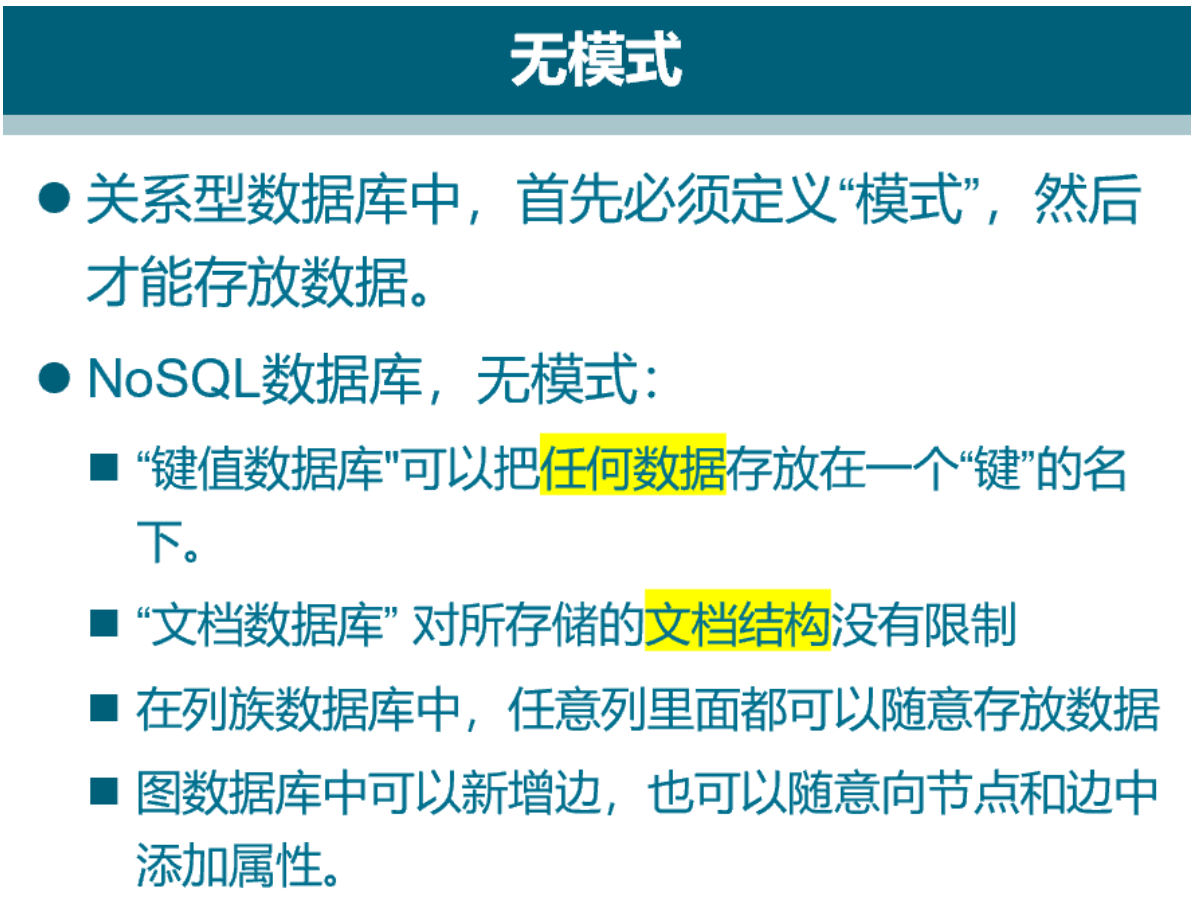
面向聚合数据库总结

1. 共同点
    1. 都使用聚合这一概念，而且聚合中都有一个可以查找其内容的**索引键。**
    2. 在集群上运行时，聚合是中心环节，因为数据库必须保证将**聚合内的数据存放在同一个节点**上。
    3. 聚合是"更新"操作的最小数据单位，对事务控制来说，**以聚合为操作单元**
2. 差别
    1. 键值数据模型将聚合看作**不透明**的整体，只能根据键来查出**整个聚合**，而不能仅仅查询或获取其中的一部分
    2. 文档模型的聚合对数据库**透明**，于是就可以只查询并获取其中一部分数据了，不过，由于文档没有模式，因此在想优化存储并获取聚合中的部分内容时，数据库不太好调整文档结构
    3. 列族模型把聚合分为列族，让数据库将其视为行聚合内的一个数据单元。此类聚合的结构有某种**限制**，但是数据库可利用此种结构的优点来提高其易访问性。

图数据库

1. 图数据库的基本数据模型：由边(或称"弧")连接而成的若干节点。
2. 可以用专门为"图"而设计的查询操作来搜寻图数据库的网络了。遍历：非常迅速。
3. 与面向聚合数据库的明显差异：重视数据间的关系。

SQL与NoSQL最大的区别



更改数据有差别，且无模式更易处理**格式不一致的数据**

问题：

1. 存在“**隐含模式**”。在编写数据操作代码时，对数据结构所做的一系列假设
2. **不同程序**访问相同数据库比较麻烦，因为无模式本质是将模式交给访问其数据的应用程序代码来处理。好在有两种解决方法：
    1. 使用应用程序数据库，并使用Web Services、SOA等完成**集成**
    2. 在聚合中为不同应用程序明确**划分出不同区域**。在文档数据库中，可以把文档分成不同的区段。在列族数据库，可以把不同的列族分给不同的应用程序。

66 分布式模型

1. 数据分布有两条路径:复制与分片。既可以在两者中选一个来用，也可以同时使用它们。
    1. “分片”则是将不同数据存放在不同节点中
    2. “复制”就是将同一份数据拷贝至多个节点;
        - “主从式”和“对等式”
2. 单一服务器：最简单的分布形式**根本不分布。将数据库放在一台电脑中**，让它处理对数据存储的读取与写入操作。
3. 分片：把数据的各个部分存放于不同的服务器中，以此实现横向扩展。
    1. 在理想情况下，不同的服务器节点会服务于**不同的用户**。每位用户只需与一台服务器通信，并且很快就能获得服务器的响应。**网络负载相当均衡**地分布于各台服务器上。
    2. 为达成目标，必须保证需要**同时访问的那些数据都存放在同一节点上**，而且节点必须排布好这些数据块，使访问速度最优
        1. 若使用面向聚合的数据库，可以把**聚合**作为分布数据的单元。
        2. 地理：距离近一点
        3. 负载均衡：聚合数据均匀分布在各个节点

4. 聚合有序放置：依次读取的聚合放一起
5. 如果采用应用程序的逻辑实现分片，会变复杂。所以应该使用NoSQL数据库提供的**“自动分片”**功能。

3. 分片可以同时提升读取与写入效率。
4. 分片对改善数据库的“故障恢复能力”帮助并不大。尽管数据分布在不同的节点上，但是和“单一服务器”方案一样，**只要某节点出错，那么该分片上的数据就无法访问了。**反而会**降低**数据库的错误恢复能力。

4. 主从复制：

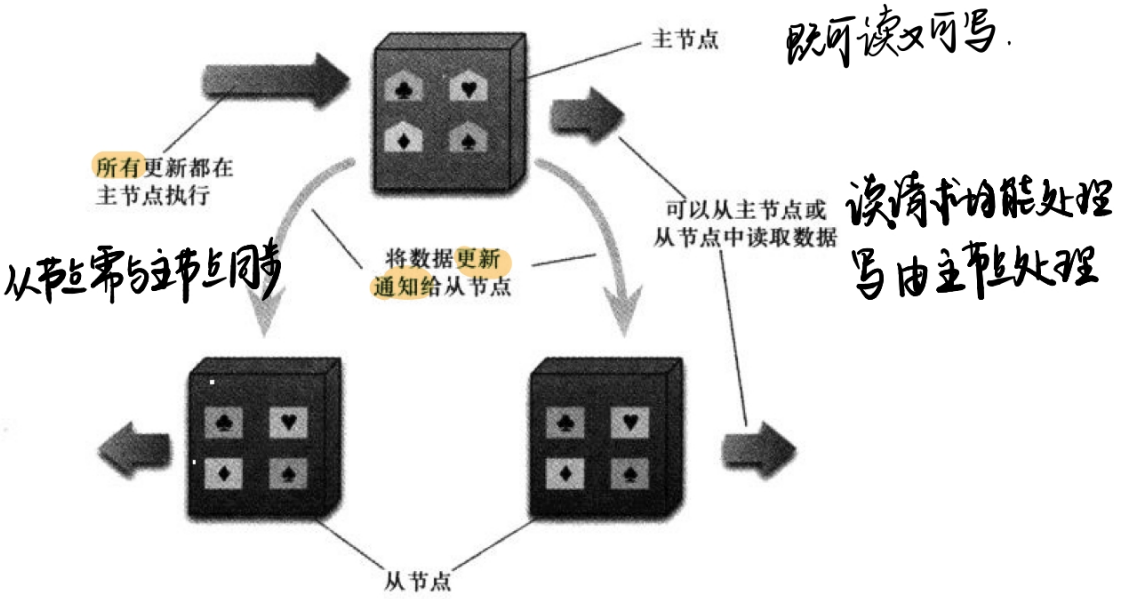


图 4.2　数据由主节点复制到从节点。所有写入请求都由主节点处理，读取请求可以由主节点处理，也可以交给从节点

- 在需要**频繁读取**数据集的情况下，“主从复制”有助于提升数据访问性能
- “主从复制” 可以增强“读取操作的**故障恢复能力**”。主节点出错时可以立即指派**新节点**。
- 缺陷：数据不一致性。

5. 对等复制：

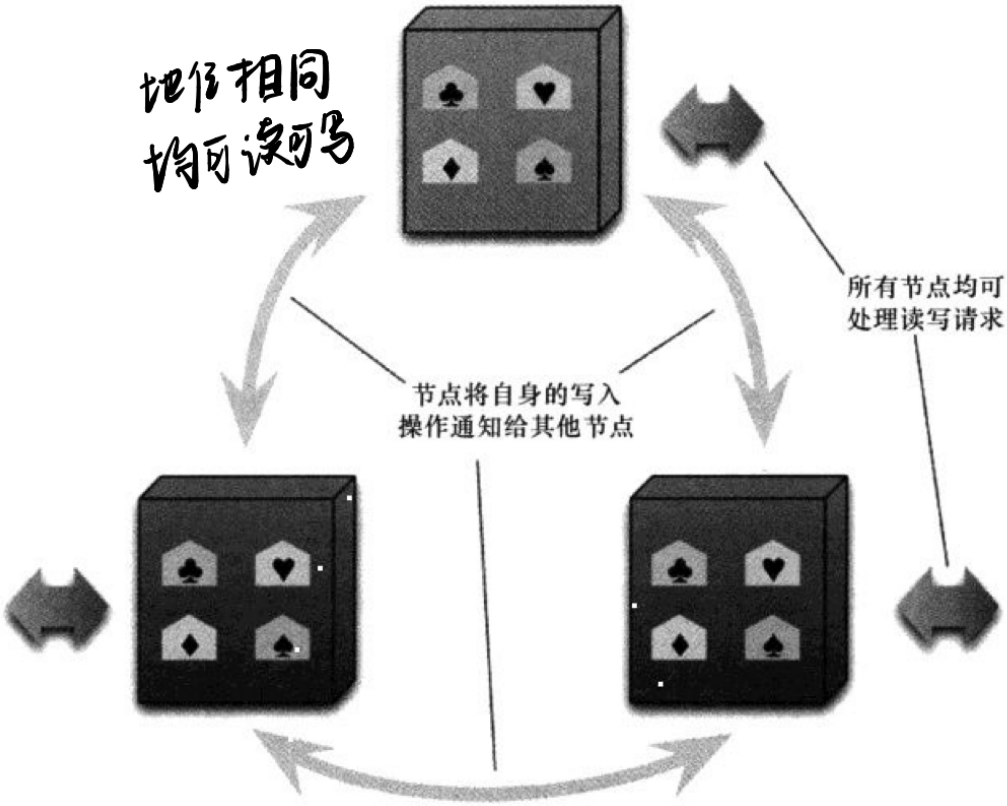


图 4.3 “对等复制”方案的所有节点都可以执行数据读写操作

- 麻烦：数据的一致性。两个结点可以同时处理写入操作，可能出现“写入冲突”。但可以解决。

6. **结合“主从复制”与“分片”**：整个系统有多个主节点，然而对每项数据来说，**负责它的主节点只有一一个。**同一个节点既可以做某些数据的主节点，也可以充当其他数据的从节点，此外，也可以指派全职的主节点或从节点。
7. **结合“对等复制”与“分片”**：在采用“对等复制”方案时，一开始可以用“3”作为**复制因子**，也就是把每个分片数据放在3个节点中。一旦某个节点出错，那么它上面保存的那些分片数据会由其他节点重建。

67 分布式模型中的一致性

1. 更新一致性
   - 当两个客户端试图同时修改一份数据时，会发生“写入冲突”。而当某客户端在另一个客户端执行写入操作的过程中读取数据时，则会发生“读写冲突”。
   - 悲观方式：写入锁
   - 乐观方式：**条件更新**，即任意客户在执行更新操作之前，都要先测试数据的当前值和其上一次读入的值是否相同
2. 读取一致性
   - **事务**可以避免读写冲突造成的逻辑不一致。
   - “图数据库”常常和关系型数据库一样，也支持ACID事务。所以可以用事务解决。
   - 面向聚合的数据库通常支持“原子更新”,但仅限于单一聚合内部。所以**内部可以保持逻辑一致性，但各聚合之间不可以。**
3. 复制一致性：要求从不同副本中读取同一个数据项时，所得到的值相同。

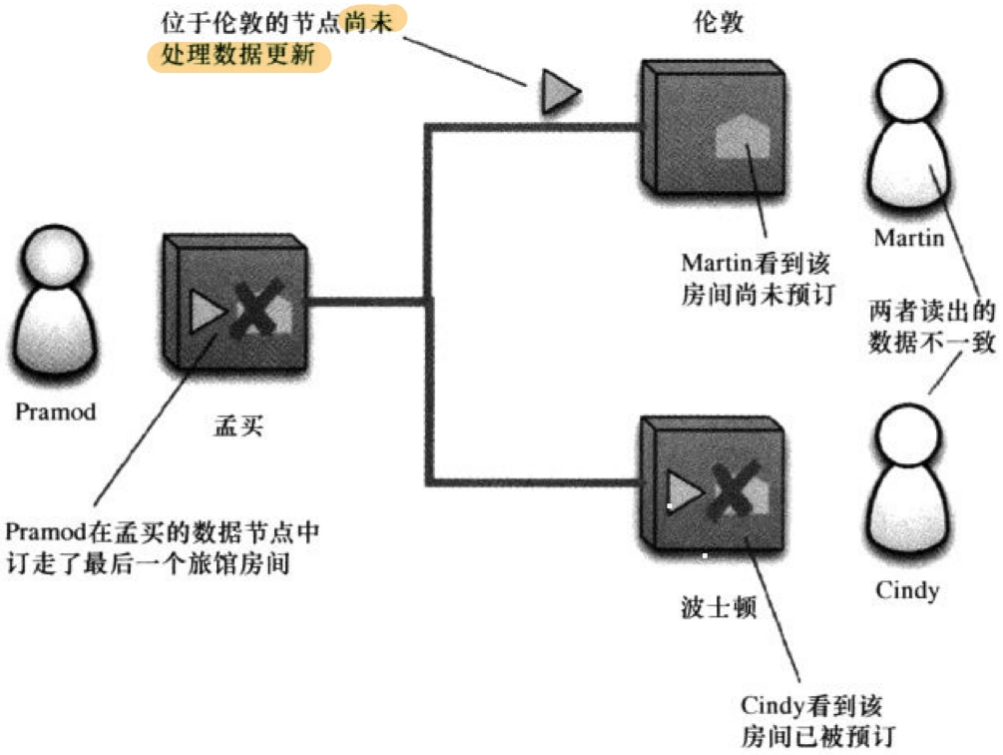


图 5.2　举例说明“复制不一致”问题

- 若写入操作已经传播至所有节点，则此刻的数据库就具备“**最终一致性**”
- 复制不一致性带来的“不一致窗口”，在考虑网络环境后，会比单一节点导致的“不一致窗口”长的多

4. 照原样读出所写内容的一致性：在执行完更新操作之后，要能够**立刻看到新值**。在具备“最终一致性” 的系统中，可以提供“**会话一致性**”
    - 使用“黏性会话“
    - 使用“版本戳”

68 放宽“一致性”和“持久性”约束

1. 使用“事务”达成强一致性。引入放松“隔离级别”的功能，以允许查询操作读取尚未提交的数据。
    1. 会降低性能

CAP定理

1.

使用事务保障“一致性”

- 使用“事务”达成强一致性
- 引入放松“隔离级别”（isolation level)的功能，以允许查询操作读取尚未提交的数据。
  - 读未提交，一个事务可以读取另一个未提交事务的数据。脏读
  - 读已提交，一个事务要等另一个事务提交后才能读取数据。不可重复读
  - 可重复读，在开始读取数据（事务开启）时，不再允许修改操作。幻读
  - 可串行化，事务串行化顺序执行。严格一致性，效率是一个问题

2. CAP定理是需要放宽一致性约束的原因。同时满足两个即可。
    1. 一致性
    2. 可用性：如果客户可以同集群中的某个节点通信，那么该节点就必然能够处理读取及写入操作。

       可用性定义为“系统中某个**无故障节点**所接收的每一条请求， 无论成功或失败，都必将**得到响应。**”
    3. 分区耐受性：如果发生通信故障，导致整个集群被分割成多个无法互相通信的分区时(这种情况也叫“ 脑裂”)，集群仍然可用。如下图，中间被分割，两侧均可用。

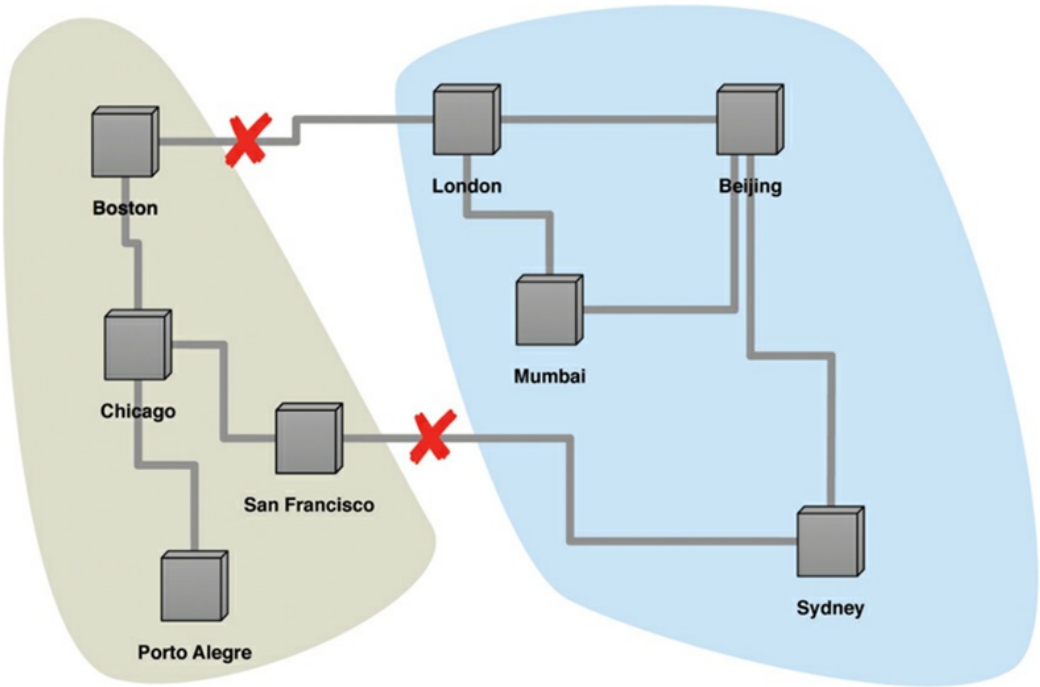


Figure 5.3. With two breaks in the communication lines, the network partitions into two groups.

3. CA系统：大多数关系型数据库
    1. 无法保证“分区耐受性”，这使得一旦“分区”发生，所有节点必须停止运作
4. 尽管“CAP定理”经常表述为“三个属性中只能保有两个”，实际上当系统可能会遭遇“分区”状况时(比如分布式系统)，需要在**“一致性”与“可用性”**之间进行权衡。不是二选一！

    1. 改善可用性：不用对等式而用主从式；或者直接都接受
5. BASE属性：基本可用，柔性状态，最终一致性
6. 对于一个读写请求，越多的结点参与，一致性就越高，但响应时间越长，延迟越高，可用性越低
7. 如一个节点处理完更新操作之后，在更新数据尚未复制到其他节点之前就出错了，那么则会发生“**复制持久性**” 故障

   解决：不重新指派新的主节点；确保主节点在收到某些副本对更新数据的确认之后，再告知用户它已接纳此更新

69 仲裁

处理请求所的节点越多，避免“不一致”问题的能力就越强，要想保证“强一致性”， 需要使用多少个节点才行?想要保证能够读到最新数据，必须与多少个节点联系才行?

对于对等式分布模型：

1. 写入仲裁：如果发生两个相互冲突的写入操作，那么只有其中一个操作能为超过半数的节点所认可，**W>N/2** 。即，**参与写入操作**的节点数(W)，必须超过副本节点数(N)的一半。副本个数又称为“复制因子”
2. 读取仲裁：只有当**R+W>N**时，才能保证读取操作的“强一致性”。其中，执行读取操作时所需联系的节点数(R)，确认写入操作时所需征询的节点数(W)，以及复制因子(N)
    1. 即使写入不具备强一致性，该公式也能保证读取具有强一致性
    2. 如果通信读取的数据一样，肯定是没问题的。

对于主从式分布模型：对主节点读写准没错。

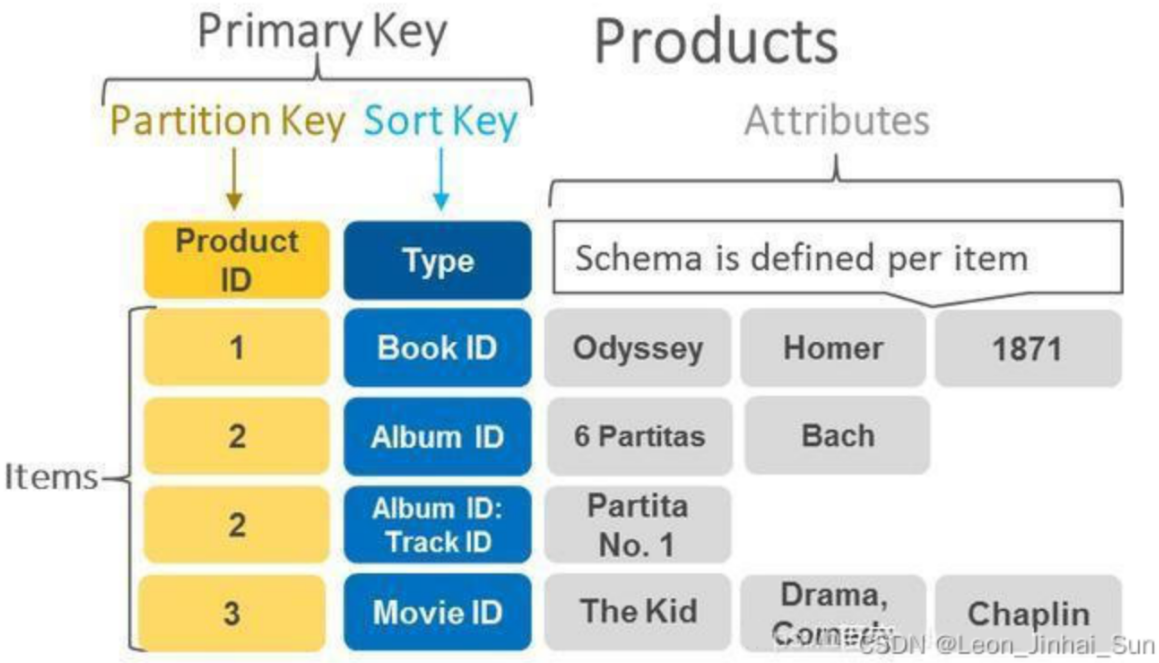
70 版本戳x

1. “条件更新”：乐观离线锁是条件更新的一种形。客户端执行操作时，将重新读取“商业活动”所依赖的信息，并检测该信息在首次读取之后是否一直没有变动，若**一直未变**，则将其展示给用户。
2. 通过保证数据库中的记录都有某种形式的“版本戳”实现“乐观离线锁”。
    1. 版本戳是一个字段，每当记录中的底层数据改变时，其值也随之改变。
    2. 读取数据时可以记下版本戳，这样的话，在写入数据之前，就可以先检查一下**数据版本是否已经变了。**
    3. CAS操作：数据库条件更新时比较的是值对应的版本戳。
    4. 构建方式：
        1. 使用计数器
        2. 使用GUID（全局唯一标识符），也就是一个值很大且保证唯一的随机数。
        3. 根据资源内容生成哈希码(hash)。只要哈希键足够大，那么“内容哈希码”(content hash)就可以像GUID那样全局唯一，而且任何人都可以来生成它。
        4. 使用上一次更新时的时间戳(timestamp)。
        5. 复合版本戳
    5. 主从式模型中：从节点必须使用主节点的版本戳。
        1. 若有多个主节点：
            1. 确保所有节点都有一份“版本戳记录”
            2. 使用“时间戳”
    6. 对等式模型中：没有统一设置版本戳的地方
        1. “数组式版本戳” 由一系列计数器组成，每个计数器都代表一个节点。
        2. [blue: 43，green :**54**，black: 12] 。假设绿色节点执行了一次更新操作，那么现在这个“数组式版本戳”就成了[blue: 43，green: **55**， black: 12]。

3. 若两个版本戳中都有一个计数器比对方大，那么就发生了“写入冲突”
4. 数组中可能缺失某些值，我们将其**视为0**。比如，[blue: 6, black: 2] 与[blue: 6，green: 0，black: 2]等价。
5. 只能检测，不能解决冲突！依赖领域知识。

7. 在“一致性”与延迟之间权衡时：如果偏向“一致性”，那么系统在出现“**网络分区**”现象时就无法使用反之，若要减少延迟，则必须**自己检测并处理“不一致”问题。**

71 键值数据库



1. 在键值数据库中，所存储的聚合不一定是领域对象，也可以拥有通用数据结构
2. 使用单一存储区，把所有数据都放在**一个对象**里，并将其存入单一的存储区中

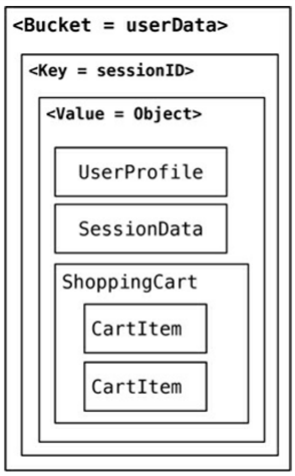


Figure 8.1. Storing all the data in a single bucket

将各类对象(也就是聚合)全部存放在一个“存储区”中，其缺点是：“存储区”中可能要存放类型不同的多个聚合，这增加了**关键字冲突**的几率。把对象名放在键名后面即可。

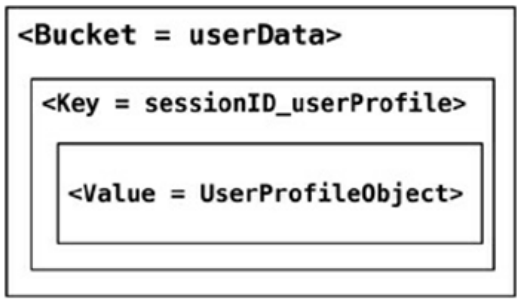


Figure 8.2. Change the key design to segment the data in a single bucket.

3. “领域存储区”来存放特定数据，可以把数据分割为对象，放在不同的存储区中
4. 键值数据库特性
    1. 一致性：分布式键值数据库，用“最终一致性模型” 实现“一致性”。在创建“存储区”时设置与一致性有关的选项。
    2. 事务：一般来说无法保证写入操作的“一致性"。用W和复制因子实现仲裁。
    3. 查询功能：关键字查询，只能获得整个value。通过某种算法生成key。
    4. 数据结构：不关心键值对里的值。
    5. 可扩展性：可用分片技术扩展。采用此技术后，键的名字就决定了负责存储该键的节点。
5. **适用案例**：
    1. 存放会话信息
    2. 用户配置信息
    3. 购物车数据
6. **不适合场合**：
    1. 数据间关系
    2. 含有多项操作的事务
    3. 查询数据
    4. 操作关键字集合

72 文档数据库

1. 文档数据库中，放在同一“集合”内的各文档的“数据模式”可以不同
2. 文档中可以嵌套数组等基本数据类型，也可以将“子文档” 以“子对象” 的形式嵌入主文档。
3. 由于没有“数据模式”约定，文档数据库的文档中**无需空属性**，若其中不存在某属性，就假定该属性值未设定或与此文档无关。向文档中新增属性时，**既无需预先定义，也不用修改已有文档内容。**
4. 特性：
    1. 一致性：配置副本集
    2. 事务：其写入操作要么成功，要么失败。
    3. 可用性：可以用主从式数据复制技术来增强“可用性”。副本集中有一个主节点，若干从结点。
    4. 查询功能：**可以查询文档中的数据**，而不用像键值数据库那样，必须根据关键字获取整个文档，然后再检视其内容。
    5. 可扩展性：
        1. **增加更多的“读取从节点”**，将读取操作导引至从节点上，这样就可以扩展数据库应对**频繁读取**的能力了。
        2. 如果想扩展写入能力，可以**把数据“分片”** 。
        3. 可以把**每个"分片”都做成“副本集”**，以提高其读取效率。
5. 适用案例：

1. 事件记录
2. 内容管理系统及博客平台
3. 网站分析与实时分析
4. 电子商务应用程序

6. 不适合场合：
    1. 包含多项操作的复杂事务
    2. 查询持续变化的聚合结构

73 列族数据库

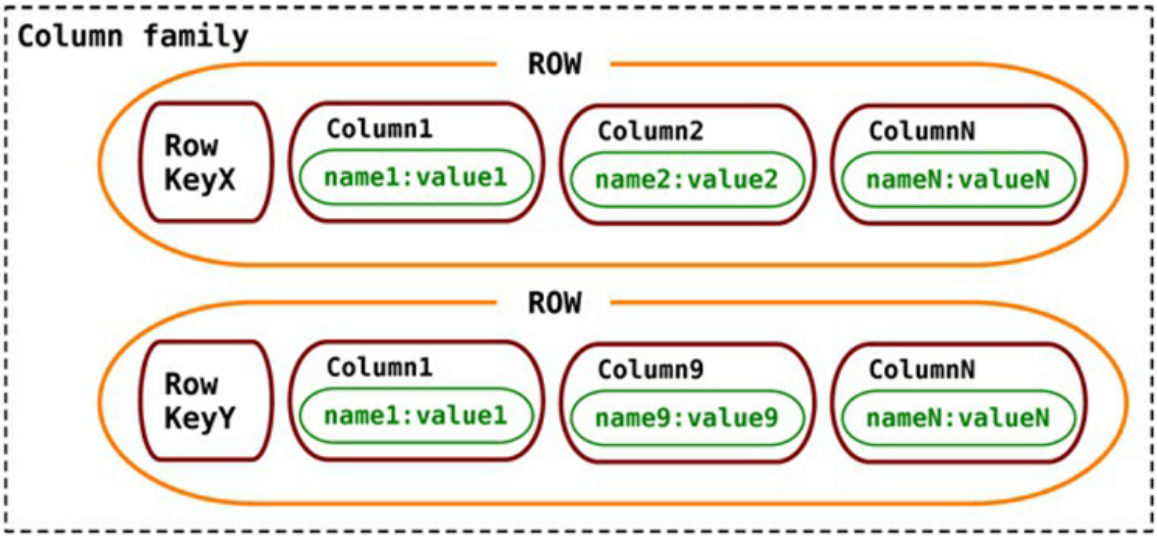


Figure 10.1. Cassandra's data model with column families

1. Cassandra的基本存储单元为"列"，列由一个"名值对"组成，其中的名字也充当关键字。
2. 每个键值对都占据一列，并且都存有一个"时间戳"值。
3. 特性：
    1. 一致性：Cassandra收到写入请求后，会先将待写数据记录到"提交日志"中，然后将其写入内存里一个名为"内存表"的结构中。**写入操作在写入"提交日志"及"内存表"后，就算成功了。**
    2. 事务：Cassandra 的写入操作在"行"级别是"原子的"。根据某个给定的行键向行中**插入或更新多个列**，将算作一个写入操作，它要么成功，要么失败。
    3. 可用性：没有主节点。(R+W) >N。
    4. 查询功能：由于Cassandra没有功能丰富的查询语言，所以在设计其数据模型时，应该优化列与列族，以提升数据读取速度。比如排序，选行键。
    5. 可扩展性：在已有的Cassandra集群中扩展，也就意味着增加更多节点。
4. 适用案例：
    1. 事件记录
    2. 内容管理系统与博客平台
    3. 计数器
    4. 限期使用
5. 不适合场合：
    1. 需要以"ACID事务"执行写入及读取操作的系统。
    2. 在开发早期原型或刚开始试探某个技术方案时。

74 图数据库

1. 图数据库可存放实体及实体间关系。实体和关系都带有属性。
2. 特性：
    1. 一致性：通过**事务**来保证"一致性"。

2. 事务：兼容ACID事务。**修改节点或向现有节点新增关系前**，必须先启动事务。读取操作可不通过事务执行。
3. 可用性：支持“副本从节点”。
4. 查询功能：可以使用查询语言，可以建立索引。遍历图时，可以通过查询索引找出起始节点。DFS图、查找路径是优点。
5. 可扩展性：因为不面向聚合而是面向关系，所以分片比较难。把相关节点放在同一台服务器中，遍历图时会更方便些。若数据集太大，导致多节点复制不太现实，那么可用“领域特定知识”在应用程序端对其分片。

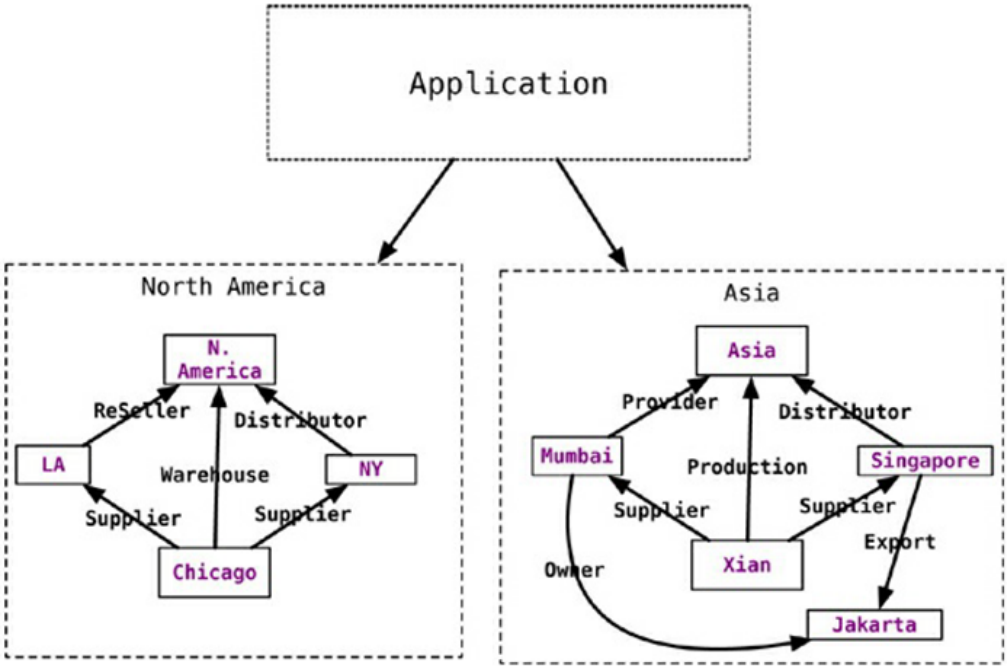


Figure 11.3. Application-level sharding of nodes

3. 适用案例：
    1. 互联数据
    2. 安排运输路线、分派货物和基于位置的服务
    3. 推荐引擎
4. 不适合场合：
    1. 图数据库在更新全部或某子集内的实体时也许不适用。
    2. 即便数据模型适合问题领域，某些图数据库可能也无法处理那么大的数据量，尤其在执行“全局图操作”时更是如此。